Reorganizing cuADMM — revision 2

An ADMM-first, cone-extensible architecture with first-class free variables, memory-aware PSD-projection strategies, and direct (and later preconditioned) linear solvers

Reorganization plan for the cuADMM developers

October 20, 2026

Baseline: `main` at `2d8a755` (submodule `psd_projection` at `b566590`).

Revision 2 answers the review of October 20, 2026 [19] and the σ-rule study of October 19 [18].

Abstract

cuADMM is a GPU solver for large multi-block conic programs, built around the first-order method of [1] and the factorization-free PSD projection of [2]. Its numerics are sound and well tested, but nearly all logic lives in one class (`SDPSolver`) whose `solve()` method spans about 1,080 lines; configuration is spread over positional arguments, public fields and preprocessor constants; and several algorithmic options are compiled out or unreachable. This plan reorganizes the code into a layered library in which (i) plain ADMM with step length $\tau \in (0, 2)$ is the default and sGS-ADMM is a runtime option, justified by the equivalence result of [3]; (ii) the penalty parameter follows a residual-balancing rule that *stops* changing (a freeze), so that the fixed-σ theory applies; (iii) every cone is a plug-in, with free variables treated as a first-class cone; (iv) the PSD projection is a catalog of named strategies whose memory model differs by block size—batched and throughput-oriented for small and medium blocks, one block at a time through a shared arena for large blocks; and (v) the normal-equation solve is an interface that supports sparse and dense direct methods now and leaves room for inexact, preconditioned iterative solves. The plan lists the defects found so far, the target architecture, a single table of options, a test strategy, and an incremental migration (a pre-step and phases 0–10) with acceptance criteria.

Contents

1 Summary **3**

2 Background and the three papers **4**

3 Mathematical specification **6**

3.1 Problem class . . . 6

3.2 ADMM, sGS-ADMM and their equivalence . . . 6

3.3 Cone step for a product cone . . . 7

3.4 Scaling and KKT residuals . . . 7

3.5 The penalty parameter . . . 8

4 Assessment of the current code **9**

4.1 What must be preserved . . . 9

4.2 Structural problems . . . 10
4.3 Defect register . . . 11

5 Design principles **13**

6 Target architecture **14**
6.1 Directory layout . . . 14
6.2 Naming and coding conventions . . . 16
6.3 Public API . . . 16

7 Algorithm layer **17**
7.1 The iteration loop . . . 17
7.2 Iteration schemes . . . 17
7.3 Penalty policies . . . 18
7.4 Termination, validation and the returned iterate . . . 18
7.5 Reporting . . . 19

8 Cones **19**
8.1 Interface . . . 19
8.2 Existing cones . . . 19
8.3 Adding a cone (checklist) . . . 20
8.4 Planned cones . . . 20
8.5 Scaling compatibility . . . 20

9 Free variables **20**
9.1 Mathematics . . . 20
9.2 Requirements on every component . . . 21
9.3 Tests . . . 21
9.4 Research option: enforce the free equalities in the y-step . . . 21

10 PSD cone projection **22**
10.1 Design philosophy by block size . . . 22
10.2 Interfaces . . . 22
10.3 Strategy catalog . . . 23
10.4 The large-block memory model . . . 24
10.5 Large-block pipeline and algorithms . . . 25
10.6 Small and medium tiers . . . 26
10.7 Selection policy for large blocks . . . 26
10.8 Integrating the composite polynomial filter . . . 27
10.9 PSD options . . . 28

11 Linear systems **28**
11.1 Role . . . 28
11.2 Supported methods . . . 29
11.3 Interface . . . 29
11.4 Backends . . . 30
11.5 Selection . . . 30
11.6 Inexact and preconditioned solves (reserved) . . . 30
11.7 Rank deficiency and presolve . . . 31
11.8 Benchmark protocol . . . 31

12 Problem input and output **31**

13 Scaling **34**

14 Core infrastructure **34**

15 Unified options **34**

16 Testing, benchmarking and continuous integration **36**

17 Migration plan **37**

18 Decisions **39**
18.1 Adopted from the review . . . 39
18.2 Still open . . . 40

A Response to the review of October 20 **40**

B Mapping from current files to the new layout **42**

C Notation and code names **43**

1 Summary

Decisions.

D1. ADMM is the default algorithm. Plain two-block ADMM with multiplier step length $\tau = 1.618$; $\tau = 1.918$ stays available as an aggressive setting, and any $\tau \in (0, 2)$ is accepted. sGS-ADMM becomes a *runtime* option (today it is removed at compile time by `CUADMM_PLAIN_ADMM_ONLY` in the CMake library). Both methods iterate the same fixed-point map in the z-variable [3] (Section 3.2).

D2. The penalty rule must stop changing σ. The default policy balances the residuals early and then freezes σ, so that its total variation is finite and the fixed-σ convergence theory applies (Section 3.5). The historical rule is restored under its historical name `legacy_adaptive`, and

the October 13 rule now on `main` becomes `residual_balancing`.

D3. Linear systems: direct methods now, iterative methods later. Sparse direct (cuDSS on the GPU, CHOLMOD on the CPU) and dense direct (cuSOLVER) backends sit behind one interface that already carries the hooks for inexact, preconditioned iterative solves (Section 11). cuDSS is the intended default once the benchmark of Section 11.8 confirms it.

D4. Composite polynomial filtering is integrated directly, in FP16 only, as the default warm-up of large blocks with $n \geq 5000$; the `psd_projection` submodule is retired after parity tests pass (Section 10.8).

D5. Cones are plug-ins, and free variables are first class. Free, nonnegative and PSD blocks are ported to one `Cone` interface; every component states what it does for free variables (Section 9).

D6. Memory-aware PSD projection. Small and medium blocks are processed in batches for throughput; large blocks are processed one at a time through one shared arena sized for the largest block, which cuts the large-tier memory from about $7\sum_b n_b^2$ to about $5\,n_{\max}^2$ doubles (Section 10).

D7. Software-engineering standards. One options object with one table of defaults, explicit ownership of GPU resources, no allocation in the iteration loop, deterministic by default, consistent naming, option and policy names that never silently change meaning, and a migration in which every structural step reproduces the current iterates bit for bit.

D8. Data formats. A native HDF5 format for very large problems, and readers for SDPA, SeDuMi, SDPT3 and MOSEK MATLAB data and the legacy text files, all producing one `ProblemData` (Section 12).

What changed since revision 1. Every recommendation of the October 20 review [19] is adopted or answered; Appendix A maps each one to the section that addresses it. The main changes:

- a new penalty plan (balance, then freeze) with a separate decision step, Phase 4b;
- a pre-step ("Pre-0") that restores `legacy_adaptive`, names the edited rule, and records the policy in the summary;
- a rewritten PSD section with an explicit, measured memory model for large blocks;
- the composite filter reduced to FP16, on by default for large blocks with $n \geq 5000$;
- a new section on input and output formats, including MOSEK's MATLAB format used by SPOT;
- a new section on free variables;
- interface hooks and a plan for preconditioned iterative linear solves;
- a smaller first milestone, Phases 8–9 deferred, and corrected facts and citations.

2 Background and the three papers

- **cuADMM** [1] introduced the solver: sGS-ADMM for multi-block SDPs, PSD projection by cuSOLVER (batched Jacobi for many small blocks, QR-based EVD on streams for larger ones),

Step	Content	Iterates
Pre-0	Restore `legacy_adaptive` (9bb4ba5); add `residual_balancing`; policy in the summary; housekeeping	legacy: as 9bb4ba5
0	Safety net: fixed-σ baselines, per-machine baselines, published golden generator, GPU gate scripts	baseline
1	Cleanup and one-line fixes (B1, B2, B6, B12, B15–B17, B21, B22)	unchanged[†]
2	`SolverOptions`; CLI and MATLAB rebuilt on it	unchanged
2b	Input and output formats: HDF5, SDPA, SeDuMi, SDPT3, MOSEK, TXT; `cuadmm-convert`	unchanged (TXT)
3	Thin core: `GpuContext`, `ExecContext`, buffer types, unified checks	unchanged
4	Algorithm layer: schemes, penalty policies (incl. `balance_freeze`), termination, observers	unchanged
4b	Tune `balance_freeze` and make it the default: CPU screen, then GPU gates ($\leq$ 12.6 H200-hours)	by design
5	Cone layer: `Cone`, free/nonnegative/PSD cones; validator kept independent	unchanged
6a	PSD strategies as named projectors; FP16 composite filter moved in, default warm-up for $n \geq 5000$	unchanged[‡]
6b	Large-block arena, per-block streaming, SYRK products; medium/small workspace fixes	within rounding
7	Linear systems: interface, CHOLMOD port, cuDSS, dense cuSOLVER, presolve; benchmark	within tolerance
8	(deferred) Data layout: contiguous slices per cone	within rounding
9	(deferred until needed) Second-order cones	new tests
10	Documentation, examples, release	—

Table 1: Migration overview (Section 17). The first milestone is Pre-0 to 4b. [†]B2 only removes a possible nondeterminism; B22 changes only the composite path. [‡]Exact and LOBPCG paths. The composite path is not reproducible before the B8 fix and is accepted on accuracy; turning on the default warm-up for blocks with $n \geq 5000$ is a deliberate change.

and a hybrid CPU–GPU normal-equation solve with CHOLMOD, building on the trajectory optimization pipeline of [4]. Its experiments target sparse moment relaxations of contact-rich planning and multi-block SDPs from Mittelmann's collection.

- **Composite polynomial filtering** [2] replaces the eigenvalue decomposition of a large dense block by a short composition of odd degree-5 polynomials evaluated with GEMMs in FP16 or FP32, after scaling the spectrum with a Lanczos estimate of the norm. It is designed as a low-precision *warm start* inside ADMM, followed by FP64 projections, and notes that ADMM quickly enters a low-rank regime [5] in which LOBPCG becomes attractive.
- **Afterimage slow regions** [3] studies first-order methods for linear conic programs as fixed-point iterations. Its Table 1 and Appendix A.2 show that ADMM and sGS-ADMM induce the *same* fixed-point map in the auxiliary variable z, which is the basis of decision D1.

Two internal documents shaped this revision: the σ-rule study of October 19 [18], which replicated the solver on the CPU decision for decision against 13 GPU runs, and the review of revision 1 of October 20 [19]. Numbers quoted from them are labelled as such.

3 Mathematical specification

This section fixes the mathematics the code must implement; class and function names in later sections refer to the symbols introduced here (Appendix C).

3.1 Problem class

Let $\mathbb{X} = \mathbb{X}_1 \times \cdots \times \mathbb{X}_B$ and $\mathcal{K} = \mathcal{K}_1 \times \cdots \times \mathcal{K}_B$ with each $\mathcal{K}_\beta$ a closed convex cone in $\mathbb{X}_\beta$. cuADMM solves the primal–dual pair

$$\min_{X \in \mathbb{X}} \langle C, X \rangle \ \text{ s.t. } \ \mathcal{A}X = b,\ X \in \mathcal{K}, \qquad \max_{y \in \mathbb{R}^m,\ S \in \mathbb{X}} \langle b, y \rangle \ \text{ s.t. } \ \mathcal{A}^* y + S = C,\ S \in \mathcal{K}^*. \tag{1}$$

Today $\mathcal{K}_\beta \in \{\mathbb{S}^n_+,\ \mathbb{R}^n_+,\ \mathbb{R}^n\}$ (block codes `s`, `l`, `u`), with dual cones $\mathbb{S}^n_+$, $\mathbb{R}^n_+$ and $\{0\}$. A *free* block ($\mathcal{K}_\beta = \mathbb{R}^n$) leaves X_β unrestricted and turns the dual constraint on its rows into the equality $\mathcal{A}^*_\beta y = C_\beta$ (Section 9). PSD blocks are stored in svec form (upper triangle, column by column, off-diagonal entries multiplied by $\sqrt{2}$), so $\langle \cdot, \cdot \rangle$ is the Euclidean inner product on the stacked vector.

3.2 ADMM, sGS-ADMM and their equivalence

ADMM (default). With penalty $\sigma > 0$ and step length $\tau \in (0, 2)$, one iteration is

$$y^{k+1} = (\mathcal{A}\mathcal{A}^*)^{-1}[\sigma^{-1} b - \mathcal{A}(\sigma^{-1} X^k + S^k - C)], \qquad \text{(y-step)} \tag{2}$$

$$S^{k+1} = \Pi_{\mathcal{K}^*}(C - \mathcal{A}^* y^{k+1} - \sigma^{-1} X^k), \qquad \text{(cone step)} \tag{3}$$

$$X^{k+1} = X^k + \tau\sigma(\mathcal{A}^* y^{k+1} + S^{k+1} - C). \qquad \text{(multiplier step)} \tag{4}$$

This is the classical three-step ADMM for SDP [8], exactly (12a)–(12c) of [3] and (24a)–(24c) of [2] with τ, and what `SDPSolver::solve` executes when `switch_admm = 0`.

sGS-ADMM (option). sGS-ADMM [6, 1] inserts a second y-step after the cone step: $y^{k+1/2}$ from (2), S^{k+1} from (3) with $y^{k+1/2}$, then $y^{k+1} = (\mathcal{A}\mathcal{A}^*)^{-1}[\sigma^{-1} b - \mathcal{A}(\sigma^{-1} X^k + S^{k+1} - C)]$, then (4). It costs one extra normal-equation solve per iteration.

Equivalence [3]. Let $P_\mathcal{A} = \mathcal{A}^\dagger \mathcal{A}$ with $\mathcal{A}^\dagger = \mathcal{A}^*(\mathcal{A}\mathcal{A}^*)^{-1}$, $P^\perp_\mathcal{A} = I - P_\mathcal{A}$, and define $z^k := P^\perp_\mathcal{A} X^k + \mathcal{A}^\dagger b - \sigma P_\mathcal{A} S^k - \sigma P^\perp_\mathcal{A} C$. Under $\mathcal{A}\mathcal{A}^* \succ 0$, $\sigma > 0$ and $\tau \in (0, 2)$, both methods generate $z^{k+1} = T(z^k)$ with the same map

$$T(z) = z - P_\mathcal{A}[\Pi_\mathcal{K}(z) - \mathcal{A}^\dagger b] - \tau P^\perp_\mathcal{A}[-\Pi_{\mathcal{K}^*}(-z) + \sigma C] \tag{5}$$

([3, Table 1, eqs. (13) and (21)]). They differ only in how (X, y, S) is recovered from z ([3, eqs. (14) and (22)]), hence in the KKT residuals one observes. At fixed σ the map T is α-averaged with $\alpha = \max\{1, \tau\}/2$ in the metric $G = P_\mathcal{A} + \tau^{-1} P^\perp_\mathcal{A}$, which is why every $\tau \in (0, 2)$ is admissible [3]; convergence of the full sequence is shown in [7]. The review found the equivalence to hold numerically to about 10^{-13} [19].

Step length and the residual vote. The multiplier step gives $\mathcal{A}X^{k+1} - b = (1-\tau)(\mathcal{A}X^k - b) + \tau s^{k+1}$, where $s^{k+1} = \sigma\mathcal{A}(S^{k+1} - S^k)$ is the ADMM dual residual [18]. The primal residual that the penalty rule watches is therefore a filtered signal whose gain on sign-alternating components is $\tau/(2-\tau)$: 4.24 at $\tau = 1.618$ and 23.4 at $\tau = 1.918$. A larger τ makes the residual vote of Section 3.5 noisier, so τ is chosen first at fixed σ and the penalty rule is re-screened at the chosen τ (Q4 in Section 18).

Consequences for the design

- One iteration engine with an `IterationScheme` strategy (`Admm`, `SgsAdmm`); no compile-time switch.
- τ is a validated option in $(0, 2)$, default 1.618. The hidden τ rules of today's sGS branch ($\tau = 1.95$, reduced to $\max(1.618, \tau/1.1)$ when $\eta_d < \text{tol}$) are removed.
- The derivations of (13) and (21) in [3] use only the definition of z^k, so a mid-run switch between the two schemes at fixed σ continues the same z-orbit: the hybrid mode is a pure recovery choice. Whether to keep it is decided after a plain-ADMM run with $\tau = 1.95$ (Q5).
- No penalty rule reads sGS residuals: the sGS primal residual contracts by exactly $|1-\tau|$ per iteration whatever the progress, and in the September campaign σ climbed to 800–10^6 on every stalled problem [19].

3.3 Cone step for a product cone

The cone step (3) decomposes block by block, $\Pi_{\mathcal{K}^*}(Z) = (\Pi_{\mathcal{K}_1^*}(Z_1), \ldots, \Pi_{\mathcal{K}_B^*}(Z_B))$:

- PSD: $\Pi_{\mathbb{S}^n_+}(Z) = Q\max(\Lambda, 0)Q^\top$ for $Z = Q\Lambda Q^\top$ (self-dual);
- nonnegative orthant: $\max(Z, 0)$ entrywise (self-dual);
- free block: $\mathcal{K}^* = \{0\}$, so $S_\beta = 0$.

For cones that are not self-dual, Moreau's decomposition $Z = \Pi_{\mathcal{K}}(Z) - \Pi_{\mathcal{K}^*}(-Z)$ gives $\Pi_{\mathcal{K}^*}(Z) = Z + \Pi_{\mathcal{K}}(-Z)$, so a cone may implement whichever projection is cheaper. The same identity gives the PSD projection from the negative side, $\Pi_{\mathbb{S}^n_+}(Z) = Z + \Pi_{\mathbb{S}^n_+}(-Z)$, which the large-block strategies use when the negative part has lower rank (Section 10.5).

3.4 Scaling and KKT residuals

The solver works on a scaled problem (on by default; Section 13):

1. rows of $\mathcal{A}$ and entries of b are divided by $\nu_i = \max(1, \|A_i\|_2)$, and y_i is multiplied by ν_i;
2. with $b_s = 1 + \|\tilde{b}\|_2$ and $C_s = 1 + \|C\|_2$: $b \leftarrow b/b_s$, $X \leftarrow X/b_s$, $C \leftarrow C/C_s$, $S \leftarrow S/C_s$, $y \leftarrow y/C_s$.

The penalty σ lives in scaled units: on the original data it corresponds to $\sigma\, b_s/C_s$. Termination uses the residuals of [1, eqs. (12)–(13)] in original units,

$$\eta_p = \frac{\|\mathcal{A}X - b\|}{1 + \|b\|}, \quad \eta_d = \frac{\|\mathcal{A}^* y + S - C\|}{1 + \|C\|}, \quad \eta_g = \frac{|\langle C, X\rangle - \langle b, y\rangle|}{1 + |\langle C, X\rangle| + |\langle b, y\rangle|}, \quad \eta = \max(\eta_p, \eta_d, \eta_g),$$

and, at termination and in validation mode, the cone violations of X and S.

3.5 The penalty parameter

Rules in the code today. All of them count "wins" after every iteration (w_p if $\eta_p < \eta_d$, else w_d) and act only at the check iterations $k \in \{k \le 200 : k \equiv 1\ (10)\} \cup \{200 < k \le 1000 : k \equiv 1\ (25)\} \cup \{1000 < k \le 5000 : k \equiv 1\ (50)\} \cup \{5000 < k \le 10^4 : k \equiv 1\ (200)\} \cup \{k > 10^4 : k \equiv 1\ (1000)\}$, where

$$\begin{aligned} &\text{if } w_p > \max(g,\ 1.35\, w_d): && \sigma \leftarrow \min(\sigma_{\max},\ f\sigma), \quad w_p \leftarrow 0; \\ &\text{else if } w_d > \max(g,\ 1.35\, w_p): && \sigma \leftarrow \max(\sigma_{\min},\ \sigma/f), \quad w_d \leftarrow 0. \end{aligned}$$

Since X is the multiplier of $\mathcal{A}^* y + S = C$, a larger σ weights dual feasibility more. The winner-count idiom (winner above $\max(1, \text{ratio} \cdot \text{other})$, winner-only reset) is the one used in the SDPNAL+/SuiteLasso family of codes [12]; the ratio, factor, schedule and run length here differ from those codes [18].

- `legacy_adaptive` at `9bb4ba5`: $g = 0$ (no guard), $f = 2$, $[\sigma_{\min}, \sigma_{\max}] = [10^{-6}, 10^6]$, plus the "chasing" monitor (`Monitor1`) after $k = 5000$, which could move σ to $\max(\sigma, 100)$ or $\min(\sigma, 0.01)$.
- The October 13 rule, committed in place in `2d8a755` under the same name: $g = 1$, $f = 2$, $[10^{-3}, 10^3]$, `Monitor1` off. Plan name: `residual_balancing`. An uncommitted edit of October 20 sets $f = 1.2$; it becomes the option `penalty.factor` (Pre-0).

Why neither rule is enough. Both keep checking every 1,000 iterations forever. On the GPU the old rule changed σ at 95.7% of its checks; in the CPU replica the October 13 rule changed it at 70–100% of the checks after $k = 10^4$ on nine small problems, and locked `trto1` and `buck1` into exact periodic orbits that are not solutions, while fixed $\sigma = 1$ solves both [18]. The varying-penalty convergence results that let σ move both ways require σ to become fixed after finitely many iterations [9] or its variation to be summable [10]; neither rule provides that by construction. Removing `Monitor1` removed the $\times 100$ to $\times 16{,}384$ jumps that preceded most large error spikes, but also the large-σ excursions behind the old rule's speed on `vibra1` [18].

Planned rule: balance, then freeze. Keep the residual balancing during the early, dense part of the schedule—which is where the adaptive gains on the `mater` problems come from—and freeze σ at a well-chosen value once it stops making progress:

```
policy balance_freeze                (planned default; initial parameters in Table 2)
state: sigma, w_p, w_d, step = factor, dirs = [], sigmas = [], frozen = false

reset(sigma0): sigma = clamp(sigma0, sigma_min, sigma_max); w_p = w_d = 0; dirs = sigmas = []

update(k, eta_p, eta_d):             # residuals of the new iterate, original units
    if frozen: return sigma
    if residual_weight * eta_p < eta_d: w_p += 1 else: w_d += 1
    if is_check(k):                  # the 10/25/50/200/1000 schedule
        if   w_p > max(1, win_ratio * w_d): w_p = 0; change(k, +1)
        elif w_d > max(1, win_ratio * w_p): w_d = 0; change(k, -1)
    if k >= K_f:                                          freeze(k, "K_f")
    elif k >= k_min and len(dirs) >= W and |sum(dirs[-W:])| <= net_max:   freeze(k, "ping-pong")
    return sigma

change(k, d):                        # d = +1 raises sigma, d = -1 lowers it
    if step_schedule == halve_on_reversal and dirs and d == -dirs[-1]: step = sqrt(step)
    sigma = clamp(sigma * step^d, sigma_min, sigma_max); dirs.append(d); sigmas.append(sigma)
```

```
    if step_schedule == halve_on_reversal and log2(step) < 1/16: freeze(k, "step")

freeze(k, reason):
    if len(dirs) >= W and |sum(dirs[-W:])| <= net_max:        # sigma is bouncing in a band:
        sigma = geometric_mean(sigmas[-W:])                    # freeze at the centre of the band
    sigma = clamp(bias * sigma, sigma_min, sigma_max); frozen = true; log(k, reason, sigma)
```

The ping-pong test uses the *net* movement over the last W changes rather than a count of reversals: the `trto1` orbit has the pattern $+\,+\,-\,-$, in which only half of the consecutive pairs reverse, so a reversal count would miss it, while its net movement is zero. Steady drift $(+\,+\,+\,\dots)$ has net movement W and never triggers the freeze, so σ can still travel to where it is needed.

Parameter	Initial value	Screened in Phase 4b
`factor` f	2	$\{2,\ 1.2\}$; $f = 2$ has the GPU evidence (isolated $\times 2$ steps are harmless), $f = 1.2$ settles closer to balance but travels $3.8\times$ slower
`step_schedule`	`constant`	{`constant`, `halve_on_reversal`} $(2 \to \sqrt{2} \to 2^{1/4} \to \cdots)$
`freeze_iteration` K_f	10^4	$\{10^4,\ 2 \cdot 10^4,\ 5 \cdot 10^4\}$; 10^4 had the best geometric mean of these three on four problems [18]
ping-pong W, `net_max`, $k_{\min}$	8, 2, 1000	sanity check $W \in \{6, 8, 12\}$
`freeze_value`	band centre	{last value, band centre}
`freeze_bias`	1	$\{1, 2, 4\}$: where the duality gap dominates (`vibra1`, `shmup1`), the best fixed σ is 10–$39\times$ above the rule's late average [18]
`win_ratio`, `guard`, `residual_weight`	1.35, on, 1	—
$[\sigma_{\min}, \sigma_{\max}]$	$[10^{-3}, 10^3]$, σ_0 clamped	—

Table 2: Parameters of `balance_freeze`. All of them are options of `ResidualBalancingPenalty` (Section 7.3).

What the freeze guarantees, and what it does not. The rule changes σ at most once per check before the freeze, so finitely often; after the last change the iteration is fixed-σ ADMM with $\tau \in (0, 2)$ started from the current point, whose iterates converge [3, 7]. The local linear rate under strict complementarity [5] is also a fixed-σ result. Speed is not guaranteed: in the replica, a freeze at $2 \cdot 10^4$ converged wherever any rule converged but was slower than the best rule on four of the five problems it solved [19], and the frozen value mattered more than the freeze time [18]. Hence the band-centre value, the optional bias, and the Phase-4b screen. The vote still ignores the duality gap η_g; a gap-aware vote is a research item (Section 18).

4 Assessment of the current code

4.1 What must be preserved

- The numerics: the iteration matches (2)–(4); the three cones project onto the right dual cones; the svec conventions, the permutations of the CHOLMOD factor, unscaling and the residuals in original units are correct.

- The test suite (87 tests in the plain-ADMM build), which generates problems with a known strictly complementary primal–dual solution for PSD, free and nonnegative blocks and for every projection tier, and keeps golden σ histories. On the reference workstation (2× RTX 6000 Ada, CUDA 12.9, SuiteSparse 7.6.1) 85 pass: the golden test fails by construction (B14) and the rank-deficient CHOLMOD test fails with this SuiteSparse version (B9; it passes with 7.10.1 on the reviewers' H200 [19]).
- Bitwise reproducibility of the exact paths (deterministic `CUSPARSE_SPMV_CSR_ALG2`), the external validator, the certified lower bound, the LOBPCG accept-or-fall-back logic, and the error checks on library calls (with the gaps listed as B15–B16).

4.2 Structural problems

P1. One class does everything. `SDPSolver` (`include/cuadmm/solver.h`, `src/solver.cu`, 2,230 lines at `main`) has over 200 public data members; `init()` has 23 parameters and about 590 lines, and `solve()` spans lines 1040–2119. `solve()` mixes the ADMM steps, the three PSD tiers with LOBPCG and composite filtering, the σ rules, termination, external validation, best-iterate tracking, printing, and bookkeeping for a mode that is compiled out.

P2. Projection logic at two levels. Large blocks use member functions (`large_full_eig_project`, `large_lobpcg_project`, `large_composite_project`); medium and small blocks are inline in `solve()`. The buffers, handles and streams of all tiers are solver fields.

P3. Configuration in four places. The 23 parameters of `init()`, 10 of `solve()`, public fields set before either call, and `#define` constants (tier limits 32/1000, LOBPCG thresholds, composite margins). The CLI and MATLAB re-encode defaults independently, which produced B3 and B4, and the C++ API has defaults of its own (Section 15).

P4. Dead or compiled-out code. `analyze_blk`, `get_maps`, `MatrixSizes::init` (all superseded by `BlockStructure`), `get_eig_rank_mask`, `max_dense_vector_zero_mask` and the `use_mask` path, `check_gpus`, `bscale2`/`Cscale2`, the commented-out monitor, and the sGS code that the CMake library compiles out but still threads through `solve()`.

P5. Scaling in four places. Applied in `init()`, re-applied for warm starts in `solve()`, undone at the end, and undone again on the host in `validate_current_iterate`.

P6. Ad hoc streams. The legacy default stream and non-blocking streams are mixed with manual synchronizations; B2 is a consequence.

P7. Cone-specific branches in seven live files. Conversion kernel, `solver.cu`, external validator, block structure, problem loader, CLI, MATLAB binding.

P8. Memory sized for concurrency that never happens. The medium tier allocates one eigensolver workspace per matrix although at most `--streams` of them run at once (B20). The large tier keeps three dense $n_b \times n_b$ buffers and one EVD workspace per block although its blocks are projected one at a time (B19).

P9. The PSD projection depends on a submodule whose kernels, helper names, error macros and handle conventions differ from the solver's (B1, B10, B15).

4.3 Defect register

Table 3 lists the defects and inconsistencies found so far, with the step that fixes each. B15–B29 are new in this revision.

Table 3: Defects and inconsistencies at `2d8a755`.

ID	Description	Location / evidence	Step
B1	`CUBLAS_TENSOR_OP_MATH` on the composite handle equals `CUBLAS_COMPUTE_32F_FAST_16F` for `cublasSgemm`, so the "FP32" composite filter runs at FP16 accuracy: relative error 6.6×10^{-4} / 6.5×10^{-4} instead of 5.3×10^{-6} / 6.5×10^{-6} at $n = 2000/5000$ (measured). Opt-in: no default path uses it. The test that mirrors the handle accepts 10^{-3}. The FP16 filter is bitwise identical under both modes (measured at $n = 1000$, 2000, 5000).	`src/solver.cu:909`; `test/large_projection_test.hpp:630`	1, 6a
B2	Race in `init()`: `get_normA` rewrites `At_csc.vals` on stream 0 while `At_csc` is copied into `A_csr` on non-blocking streams with no ordering; if the copy wins, $\mathcal{A}$ and $\mathcal{A}^*$ are inconsistent.	`src/solver.cu: 535, 541-543`	1
B3	Default `switch_proj_max_iter` is 0 in the CLI (low-rank analysis from iteration 2) and 5000 in MATLAB.	`src/main.cu`, `MATLAB/cuadmm_MATLAB.cu`	2
B4	Composite filtering is unreachable from the CLI and MATLAB, which hard-code `EIG_FP64` for both projection phases; the C++ API defaults to `COMPOSITE_FP16`.	`src/main.cu:649-650`; `MATLAB/cuadmm_MATLAB.cu: 485-486`; `solver.h:512`	2, 6a
B5	`CUADMM_PLAIN_ADMM_ONLY` is defined for the CMake library only, so the tests and CLI reject sGS-ADMM while the MEX builds sGS and the hybrid mode untested; the README documents the sGS options.	`CMakeLists.txt:94`; `MATLAB/CMakeLists.txt`	1, 4
B6	`sigscale = max(2 exp(-k/50000), 2)` is always 2 (dead decay).	`src/solver.cu:2059`	1
B7	LOBPCG and Lanczos allocate and free device memory on every call; `cudaFree` synchronizes the device.	`psd_projection/src/lobpcg.cu`, `lanczos.cu`	6b
B8	The Lanczos start vector is seeded from `std::random_device` and the clock, so composite runs are not reproducible.	`psd_projection/src/utils.cu:188`	6a
B9	With SuiteSparse 7.6.1, CHOLMOD reports failure on the rank-deficient $\mathcal{A}\mathcal{A}^* + 10^{-15}I$ and `factorize()` throws (`AAtFactorization.RankDeficientRangeSolves`); SPOT-type data with dependent rows may fail at setup on such systems.	`include/cuadmm/cholesky_cpu.h:168`	0, 7
B10	Two definitions of the kernel `max_dense_vector_zero_kernel` (cuADMM and submodule).	`src/kernels/dense_scalar.cu`; `psd_projection/src/utils.cu`	6a
B11	The composite damping divides the *input* of each polynomial in the code; [2, §3.4] describes rescaling the *output*. Coefficients match [2, Tables 1–2] exactly.	`composite_FP16.cu`, `composite_FP32.cu`	6a

ID	Description	Location / evidence	Step
B12	Documentation drift: `init()` documents a default σ of `2e2` (signature: 1.0); `lobpcg.h` documents `D` as $m \times m$ (a vector); `solver.h` says the logged residuals are scaled (they are in original units); README lists options the build rejects.	headers, README	1, 10
B13	LOBPCG tolerance is an absolute 10^{-5}, not tied to the stopping tolerance; the acceptance test is necessary but not sufficient (documented in the code).	`src/solver.cu: 36-43, 219-234`	6a
B14	The golden test `FixedSigma.LegacyAdaptiveAndPlainAdmmUnchanged` fails by construction: both plain-ADMM golden cases start with a σ change at $k = 1$, which the guard blocks.	`test/fixed_sigma_validation_test.hpp`; `legacy_golden.txt: 6, 476`	Pre-0
B15	`include/cuadmm/check.h` and the submodule's `check.h` share the include guard `CUADMM_CHECK_H`; whichever is included first wins, and the submodule's own `CHECK_*` macros only print.	both `check.h` files	3
B16	No `cudaGetLastError` after cuADMM's kernel launches; eight `exit(1)` calls in library I/O.	`src/kernels/*.cu`; `src/utils/io.cu`	1, 3
B17	The MEX accepts 0 eigensolver streams, and any medium block then divides by zero.	`src/solver.cu:779`	1
B18	`legacy_adaptive` silently runs the October 13 rule: the old rule cannot be selected, the published PLATO command (`--sigma-policy legacy_adaptive --sig 1`) now runs a different rule, CLI help and README still call it "historical", the summary JSON writes the same label for both rules with no commit stamp, and MATLAB users get the new rule without a way to choose.	`main.cu: 19-20, 31, 93, 408`; `README.md:46`; PLATO `PROTOCOL.md:36`	Pre-0
B19	Large-tier memory: three dense buffers of $\sum_b n_b^2$ doubles (matrices, $Q \max(\Lambda, 0)$, result) plus one `cusolverDnXsyevd` workspace *per large block* ($\approx 4.0\, n_b^2$ doubles each, measured), although large blocks are projected one at a time: about $7 \sum_b n_b^2$ doubles. One block of $n = 30{,}000$ alone takes ≈ 47 GiB of the reference GPUs' 48 GiB; one block of $n = 32{,}768$ (≈ 56 GiB) does not fit.	`src/solver.cu: 839-890, 954-964`	6b
B20	Medium-tier memory: one `Xsyevd` workspace per medium matrix (4.2–31 n^2 doubles each, measured), allocated even for groups that use the batched call; batched groups are decomposed in one call whatever their size, and the batched workspace grows linearly with the batch. 1,000 blocks of $n = 500$ take ≈ 8.9 GiB of streamed workspace (plus as much again for the batched call).	`src/solver.cu:768-837`	6b
B21	Exact-EVD size limit: on CUDA 12.9 the `cusolverDnXsyevd`/`Xsyevdx` workspace queries fail for $n > 32{,}768$ (FP64 and FP32, measured), while `BlockStructure` accepts PSD blocks up to 46,340; setup then fails with a cuSOLVER error instead of a clear message.	`matrix_sizes.h:17`; `src/solver.cu:869`	1

ID	Description	Location / evidence	Step
B22	The Lanczos value is an estimate, not a bound: it fell below $\|Z\|_2$ in 0.2% of random starts on one test spectrum (worst ratio 0.979) [19], and only non-finite composite results are rejected.	`src/solver.cu:297-305`	1
B23	Best-iterate ranking: without a validation the merit is normalized by $1 + \|b\|_2$, while DIMACS `err1` uses $1 + \|b\|_\infty$ (about 12× apart on the long-horizon problems); in 3 of 4 long-horizon runs the returned iterate's validated error was 1.19–6.1× that of the best one seen [19].	`external_validation.cu: 94-107, 215, 354`	4
B24	Certificate with free blocks: `--trace-bound R` applies R, and `--trace-bound-scale f` applies f, as an entrywise bound $\|X_u\| \le R_u$ on every free block; the lower bound is valid only if that holds, and nothing reports it per block (Section 9).	`src/main.cu:68-74`	5
B25	The history CSV keeps 10 significant digits, too few for exact replays and unit-test fixtures.	`src/main.cu:597`	4
B26	References on `main` to experiment material that is not published: the golden generator `gen_legacy_golden.cu` exists only in local commits, seven cited experiment commits are on no remote, and four references point to directories `main` lacks (one was never committed). Owner: the experiment authors.	`README.md:56`; `fixed_sigma_validation_test.hpp:10`; `legacy_golden.txt:2`; `external_validation.h:10`	Pre-0
B27	The only input format is text: about 3.5×10^6 nonzeros per second to parse (measured), COO triplets before CSC (twice the host memory), and `int` sizes (`At_nnz`, `vec_len`, `con_num`).	`src/problem.cu`; `src/utils/io.cu`; `problem.h:26-38`	2b
B28	`mosek_to_txt.m` treats every scalar variable as free (it overwrites `blx`), negates c (that is, assumes maximization) exactly when `barc` is absent, reads only `blc` (equalities assumed), and ignores `cfix`. Right for SPOT's moment and SOS files, wrong for other MOSEK problems.	`examples/mosek_to_txt.m`; `examples/utils/convert_mosek2sedumi.m`	2b
B29	`sdpt3_to_txt.m` and `sedumi_to_txt.m` reject `l` blocks (`store_blk` writes only `s` and `u`); each MATLAB example defines its own local `sdpt3_to_cuadmm`, with two different signatures.	`examples/*_to_txt.m`; `MATLAB/example_*.m`	2b

5 Design principles

1. **Layers with one-way dependencies.** Front-ends → solver facade → algorithm → components → core → CUDA libraries (Figure 1). No component knows the solver.
2. **Strategies, not branches.** Every algorithmic choice (iteration scheme, penalty policy, PSD projector, normal-equation backend, stopping rule) is an object chosen at runtime from options. No compile-time algorithm flags.
3. **One source of truth for options.** Defaults live in `SolverOptions`; the CLI flags and MATLAB fields are generated from, or checked against, the same table.
4. **Names never change meaning.** A behavior change gets a new option value or policy name;

a removed or renamed name is an error, never a silent alias (B18).

5. **Memory follows the access pattern.** Resources that are used concurrently are allocated per concurrent user; resources that are used one after another share one allocation sized for the largest user (Section 10). Nothing is allocated inside the iteration loop.
6. **Explicit GPU resource ownership.** Each component owns its streams, handles and workspace slices and declares cross-stream dependencies with events.
7. **Math-to-code traceability.** Names follow Section 3: `y_step`, `cone_step`, `multiplier_step`, `sigma`, `tau` (Appendix C).
8. **Deterministic by default.** Fixed seeds, deterministic SpMV, fixed reduction orders; faster nondeterministic modes are opt-in.
9. **Behavior-preserving migration.** Structural phases reproduce baseline iterates bit for bit on each pinned machine; behavior changes are isolated in labelled phases with tolerance-based acceptance.
10. **Testable components.** Each component can be driven from host data in a unit test without constructing a solver.

6 Target architecture

The library is organized in six layers (Figure 1). Each layer uses only the layers below it: the front-ends build options and call the facade; the algorithm layer drives the components through the interfaces of Sections 7–11; the components take their streams, workspaces and library handles from the core and create none of their own.

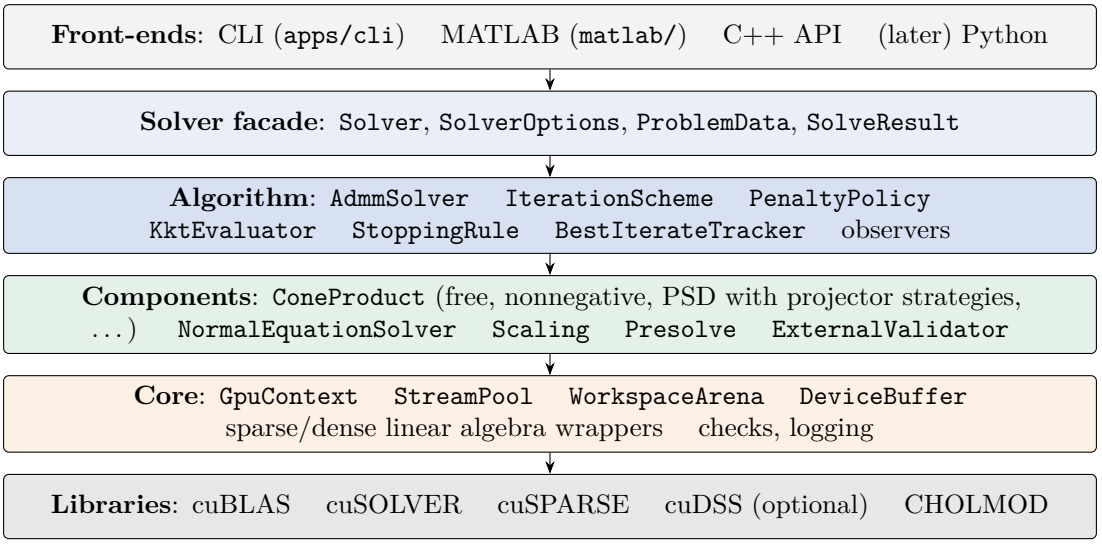


Figure 1: Layers and dependency direction. The external validator is a component of its own: a CPU implementation on the original data that shares no code with the GPU path (Section 7.4).

6.1 Directory layout

```
cuADMM/
|-- CMakeLists.txt        CUADMM_WITH_{CUDSS,HDF5,MATIO}, CUADMM_BUILD_{CLI,MATLAB,TESTS,BENCH}
     (CHOLMOD is required)
|-- cmake/                FindCUDSS.cmake, FindCHOLMOD.cmake, compiler warnings
|-- include/cuadmm/       public API only
|   |-- cuadmm.h          umbrella header
|   |-- options.h         SolverOptions (single source of defaults)
|   |-- problem.h         ProblemData, ConeSpec
|   |-- result.h          SolveResult (best and final iterate), Status, Statistics
|   `-- solver.h          Solver facade
|-- src/
|   |-- core/             GpuContext, StreamPool, ExecContext, WorkspaceArena, buffers, checks,
    logging
|   |-- linalg/           sparse matrix wrappers (CSR/CSC), SpMV, vector kernels
|   |-- problem/          BlockLayout, Presolve (dependent rows, grouping by cone), DeviceProblem
|   |-- scaling/          Scaling
|   |-- cone/
|   |   |-- cone.h, cone_product.{h,cu}, cone_registry.{h,cu}
|   |   |-- free_cone.cu, nonnegative_cone.cu         (soc/ when Phase 9 is scheduled)
|   |   `-- psd/
|   |       |-- psd_cone.{h,cu}, svec.cu
|   |       |-- batched/      small and medium tiers: batch layout, Jacobi, batched and streamed
    EVD
|   |       |-- large/        large tier: arena, LargeBlockState, RankAnalyzer, selection policy
|   |       |   `-- projectors/  FullEvd, Lobpcg, CompositeFilter
|   |       `-- kernels/      composite filters + coefficients, Lanczos estimate, LOBPCG,
    precision
|   |-- linsys/           NormalSystem, NormalEquationSolver, factory
|   |   |-- direct/       cudss/, cholmod/, dense/ (cuSOLVER)
|   |   `-- iterative/    NormalOperator, PcgSolver, preconditioners/   (research track)
|   |-- admm/             AdmmState, IterationScheme, PenaltyPolicy, AdmmSolver
|   |-- termination/      KktEvaluator, StoppingRule, BestIterateTracker, Certificate
|   |-- validation/       ExternalValidator (CPU, original data, independent of the GPU path)
|   |-- io/               readers (h5, sdpa, sedumi, sdpt3, mosek, txt), writers, detection,
    conversion report
|   `-- report/           IterationObserver: console, CSV, sigma log, JSON, checkpoints
|-- apps/cli/             cuadmm executable (argument parser driven by the option table), cuadmm-
    convert
|-- matlab/               MEX gateway and +cuadmm package (options struct)
|-- test/                 unit/, integration/, golden/, data/
|-- bench/                projection and linear-system micro-benchmarks, end-to-end suites
|-- docs/                 this plan, design notes, Doxygen configuration
|-- examples/             unchanged
`-- experiments/          unchanged (frozen records of published runs)
```

6.2 Naming and coding conventions

Item	Convention
Namespaces	`cuadmm`, `cuadmm::core`, `cuadmm::cone`, `cuadmm::cone::psd`, `cuadmm::linsys`, `cuadmm::admm`, `cuadmm::termination`, `cuadmm::validation`, `cuadmm::report`; implementation details in `detail` or anonymous namespaces.
Files	`snake_case`, named after the main class (`full_evd_projector.cu` defines `FullEvdProjector`); public headers only under `include/cuadmm`.
Types	`PascalCase` classes and structs; `enum class` with `PascalCase` values (`IterationScheme::SgsAdmm`).
Functions, variables	`snake_case`; private members end in `_` (`sigma_`).
Constants	`constexpr` with a `k` prefix (`kDefaultTau`); no `#define` for numerical parameters—those are options.
Option and policy names	lower `snake_case` strings (`balance_freeze`); unknown names are errors.
CUDA kernels	`snake_case_kernel`, file-local, launched only through a host wrapper that takes an `ExecContext` and checks the launch.
Math names	as in Appendix C: `sigma`, `tau`, `y_step`, `cone_step`, `multiplier_step`, `eta_p`, `eta_d`, `eta_g`.
Errors	one set of `CUADMM_CHECK_CUDA/CUBLAS/CUSOLVER/CUSPARSE/CUDSS` macros throwing `cuadmm::Error` with call site and context; no `exit()` in library code.
Ownership	RAII for every CUDA object; non-owning `DeviceSpan<T>` for views; no raw `cudaMalloc` outside `core`.
Style tools	`.clang-format`, `.clang-tidy`, Doxygen comments on every public type; warnings as errors in CI.

6.3 Public API

```
namespace cuadmm {

enum class ConeKind { Free, Nonnegative, Psd, SecondOrder, RotatedSecondOrder, Exponential, Power
    };
struct ConeSpec { ConeKind kind; std::int64_t size; double parameter = 0.0; };
enum class Sense { Minimize, Maximize };

struct ProblemData {                        // host data, original units, 64-bit sizes
  CscMatrix At;                             // vec_len x m, transpose of A in svec form
  SparseVector b, C;
  std::vector<ConeSpec> cones;              // in input order
  Sense sense = Sense::Minimize;            // results are reported in this sense
  double objective_offset = 0.0;            // e.g. MOSEK cfix
  io::SourceInfo source;                    // format, file, block mapping for output, conversion
    report
};
namespace io {
ProblemData read_problem(const std::string& path, const IoOptions& options);  // format from
    extension/content
void write_problem(const std::string& path_h5, const ProblemData& problem);    // native HDF5
void write_solution(const std::string& path, const ProblemData& problem, const SolveResult&
    result);  // input's convention
}

struct WarmStart { std::vector<double> X, y, S; std::optional<double> sigma; };

class Solver {
 public:
```

```
  explicit Solver(SolverOptions options);
  void setup(const ProblemData& problem);  // presolve, scale, factorize, plan cones, check
    memory
  SolveResult solve(const WarmStart* warm_start = nullptr);  // resets the penalty-policy state
  const SolverOptions& options() const;
  void add_observer(std::shared_ptr<report::IterationObserver> observer);
};

}  // namespace cuadmm
```

`SolveResult` carries both the best and the final iterate (unscaled), the residuals η_p, η_d, η_g, cone violations, the certificate quantities, the stop reason, timings per phase and component, the policy and its parameters, and the choices made at setup (normal-equation backend, PSD strategies and memory plan). The solver uses one GPU.

7 Algorithm layer

7.1 The iteration loop

After the reorganization the loop reads as the mathematics. Observers receive the residuals of the iterate after the step, paired with the σ that the next iteration will use, as the history does today:

```
SolveResult AdmmSolver::run(AdmmState& s) {
  KktResiduals r = kkt_.evaluate(s);                          // starting point, original units
  for (int k = 1; ; ++k) {
    if (const auto stop = stopping_.check(k - 1, r, s)) return finish(*stop, s);
    scheme_->iterate(s, sigma_, tau_);                        // y-step(s), cone step, multiplier
    step
    r = kkt_.evaluate(s);                                     // post-step residuals
    sigma_ = penalty_->update(k, r, sigma_);
    observers_.on_iteration(k, s, r, sigma_);
  }
}
```

7.2 Iteration schemes

```
class IterationScheme {
 public:
  virtual ~IterationScheme() = default;
  virtual std::string_view name() const = 0;
  // One iteration of (2)-(4) (ADMM) or its sGS variant.
  virtual void iterate(AdmmState& s, double sigma, double tau) = 0;
  virtual int normal_solves_per_iteration() const = 0;      // 1 for ADMM, 2 for sGS-ADMM
};
class AdmmScheme final : public IterationScheme { /* (2) -> (3) -> (4) */ };
class SgsAdmmScheme final : public IterationScheme { /* (2) -> (3) -> (2') -> (4) */ };
```

Both share the step implementations (`y_step`, `cone_step`, `multiplier_step`) in `AdmmState`. τ is validated in $(0, 2)$ at setup; an optional `scheme_switch_iteration` reproduces the hybrid mode until Q5 is decided.

7.3 Penalty policies

```
class PenaltyPolicy {
 public:
  virtual ~PenaltyPolicy() = default;
  virtual std::string_view name() const = 0;
  virtual void reset(double sigma0) = 0;                    // called by every solve(), warm
    starts included
  // Returns sigma for the next iteration; records every change for the sigma log.
  virtual double update(int k, const KktResiduals& r, double sigma) = 0;
  virtual PolicyState state() const = 0;                    // k, counters, history, frozen flag
  virtual void restore(const PolicyState& st) = 0;          // exact resume from a checkpoint
};
class FixedPenalty final : public PenaltyPolicy { };            // sigma_k = sigma_0
class LegacyPenalty final : public PenaltyPolicy { };           // 9bb4ba5, bit for bit (
    Monitor1 included)
class ResidualBalancingPenalty final : public PenaltyPolicy { };  // all options of Table 2
```

Policy name	Class	Behavior
`fixed`	`FixedPenalty`	$\sigma_k = \sigma_0$.
`legacy_adaptive`	`LegacyPenalty`	The rule of `9bb4ba5` bit for bit, for reproducing published runs and as the Phase-4b comparator. Whether to keep it after the migration is a separate decision.
`residual_balancing`	`ResidualBalancingPenalty`	The October 13 rule (`2d8a755`): guard, $f = 2$, $[10^{-3}, 10^{3}]$, no freeze. Default from Pre-0 until Phase 4b (O1).
`balance_freeze`	`ResidualBalancingPenalty`	The planned rule of Section 3.5. Becomes the default in Phase 4b, with the parameters the screen selects.

`Monitor1` survives only inside `LegacyPenalty`. Every policy writes one σ-log row per change (old and new σ, reason, η_p, η_d, η_g, counters); the summary records the policy name, its parameters and the build commit, σ_0 and the final σ in scaled and original units, the number of changes after 10^4 and after the freeze, $\sum_k |\log_2(\sigma_{k+1}/\sigma_k)|$, and the freeze iteration and reason. Unit tests drive the policies with scripted residual sequences stored at full precision (B25), so they need no GPU.

7.4 Termination, validation and the returned iterate

- `KktEvaluator` computes η_p, η_d, η_g once per iteration from the cached residual vectors and the `Scaling` object.
- Stopping rules implement `StoppingRule`: `InternalKktRule` (default), `ValidatedRule` (today's validation-aware stopping), `StrictDimacsRule`; all enforce the iteration and time limits and detect non-finite residuals.
- `ExternalValidator` stays a separate CPU implementation on the original data that uses none of the solver's GPU residuals or eigendecompositions. Each cone registers a host-side reference measure for it, kept apart from its GPU projection code.
- `Certificate` computes the cone terms of the certified lower bound through `Cone::measure` and needs an explicit bound per block where the cone requires one (free blocks: Section 9).

- `BestIterateTracker` ranks by the maximum DIMACS error when validations exist (B23), and `SolveResult` carries both the best and the final iterate.

7.5 Reporting

All bookkeeping that today lives in solver fields (`info_*` arrays, `sigma_log`, phase timers, validation history) moves to `IterationObserver` implementations: console table, CSV history with 17 significant digits, σ log, JSON summary, checkpoint writer. Observers never influence the iteration.

8 Cones

8.1 Interface

```
class Cone {
 public:
  virtual ~Cone() = default;
  virtual ConeKind kind() const = 0;
  virtual std::string_view name() const = 0;
  virtual std::int64_t dimension() const = 0;       // length of this cone's slice
  virtual bool is_self_dual() const = 0;
  virtual WorkspaceRequest workspace() const = 0;   // served by the WorkspaceArena at setup
  // Cone step (3): s = Pi_{K*}(z) on this cone's slice.
  virtual void project_dual(DeviceSpan<const double> z, DeviceSpan<double> s, ExecContext& ctx) =
     0;
  // Optional; default via Moreau: Pi_K(z) = z + Pi_{K*}(-z).
  virtual void project_primal(DeviceSpan<const double> z, DeviceSpan<double> x, ExecContext& ctx);

  // Violation measures and certificate coefficient of X (Side::Primal) or S (Side::Dual).
  virtual ConeMeasures measure(DeviceSpan<const double> v, Side side, ExecContext& ctx) = 0;
  virtual ScalingCompatibility scaling_compatibility() const = 0;
  virtual const HostReferenceMeasure& host_reference() const = 0;   // for the ExternalValidator
};
```

A `ConeProduct` owns one object per cone kind (each holding all blocks of that kind), maps slices of the vector to cones, runs their `project_dual` on separate streams, and joins them with events. A `ConeRegistry` maps input codes to factories, so the TXT reader, the MATLAB gateway and the validator never switch on cone letters.

8.2 Existing cones

Code	Primal $\mathcal{K}_\beta$	Dual $\mathcal{K}_\beta^*$	Cone step and status
`s n`	$\mathbb{S}_+^n$	$\mathbb{S}_+^n$	$Q\max(\Lambda,0)Q^\top$ through the strategies of Section 10; correct, tested for all tiers.
`l n`	$\mathbb{R}_+^n$	$\mathbb{R}_+^n$	$\max(z,0)$; correct, tested together with PSD and free blocks (a pure-LP test is added in Phase 5).
`u n`	$\mathbb{R}^n$	$\{0\}$	$s=0$; correct, tested alone and with PSD, nonnegative, small, medium blocks (Section 9).

8.3 Adding a cone (checklist)

1. Implement a `Cone` subclass with `project_dual` (or `project_primal` plus Moreau), `measure` and a host reference measure for the validator.
2. Register its input code and parameters in `ConeRegistry`.
3. Add a generator case producing a problem with a known strictly complementary solution for the cone (the pattern of `test/regression_problems.hpp`).
4. Add unit tests of the projection: idempotence, Moreau decomposition $z = \Pi_{\mathcal{K}}(z) - \Pi_{\mathcal{K}^*}(-z)$, orthogonality $\langle \Pi_{\mathcal{K}}(z), \Pi_{\mathcal{K}^*}(-z) \rangle = 0$, nonexpansiveness, and membership.
5. Declare scaling compatibility and document the cone in the input-format section.

8.4 Planned cones

- **Second-order** $\mathcal{Q}^n = \{(t, x) : \|x\| \le t\}$ (code `q`), self-dual, closed form: (t, x) if $\|x\| \le t$; 0 if $\|x\| \le -t$; otherwise $\frac{t+\|x\|}{2}(1, x/\|x\|)$. One batched kernel with a segmented reduction.
- **Rotated second-order** (code `r`) through the orthogonal map to $\mathcal{Q}^n$.
- **Exponential and power cones**: not self-dual, projections by a few Newton steps per cone; the interface separates primal and dual projections.

8.5 Scaling compatibility

The current scaling (row normalization of $\mathcal{A}$ and two scalars) is valid for every cone. A future variable scaling must map each cone onto itself: any positive diagonal scaling for free and nonnegative blocks, a congruence $X \mapsto DXD$ with positive diagonal D for PSD blocks (entrywise $d_i d_j$ in svec coordinates), and a uniform factor for second-order blocks. `scaling_compatibility()` makes this checkable.

9 Free variables

Free variables are not a corner case: SOS formulations produce large free blocks (the SOS version of the PushBox problem, `examples/SPOT/data/TXT/PushBox_N=30_SOS`, has one free block of 22,870 entries), and modeling with equality-constrained auxiliary variables or SeDuMi's `K.f` produces them too. They are handled correctly today; this section makes the handling explicit for every component, so that no part of the reorganization can drop it.

9.1 Mathematics

For a free block u, $\mathcal{K}_u = \mathbb{R}^{n_u}$ and $\mathcal{K}_u^* = \{0\}$. In (2)–(4):

- the y-step is unchanged: the columns of $\mathcal{A}$ that belong to u are part of $\mathcal{A}\mathcal{A}^*$;
- the cone step sets $S_u = 0$;
- the multiplier step updates $X_u \leftarrow X_u + \tau\sigma(\mathcal{A}_u^* y - C_u)$, with no projection;

- at a solution, the dual constraint on the rows of u is the equality $\mathcal{A}_u^* y = C_u$, and its residual is part of η_d; X_u has no cone condition, so the primal solution set can be unbounded along $\ker \mathcal{A}_u$.

9.2 Requirements on every component

Component	Requirement for free blocks	Today
Input	`u n` in `blk.txt`; MATLAB/SeDuMi `K.f`; CBF `F` when a CBF reader is added	`blk.txt` and the MEX support `u`
`FreeCone`	`project_dual`: $s = 0$; `project_primal`: identity; `is_self_dual` false	conversion kernel writes 0
y-step, multiplier step	no special case	correct
KKT residuals	η_d includes the rows of u; no primal cone violation; $S_u \equiv 0$	correct
Scaling	any positive diagonal scaling is compatible	—
Presolve	dependent rows may involve free columns; removing rows does not change the cone	—
Warm start	X_u from the file; S_u set to 0 on load	S read as given
External validator	host reference: primal violation 0; dual violation $\max \lvert S_u \rvert$; the free rows enter the dual residual and DIMACS errors	handled
Certificate	the term $-R_u \sum_i \lvert (C - \mathcal{A}^* y)_{u,i} \rvert$ needs $\lvert X_{u,i} \rvert \le R_u$; per-block bounds are explicit options, and the summary marks the bound as conditional on them	global R or f applied implicitly (B24)
Data layout (Phase 8)	free entries form one contiguous slice	interleaved

9.3 Tests

The generator already covers free blocks in `Free`, `PSD_Free`, `Free_PSD`, `PSD_Free_PSD`, `Nonneg_Free_PSD`, `Free_Free_PSD`, `SmallBatch_Free` and `Medium_Free` (each with and without LOBPCG). Added in Phase 5:

- free and nonnegative blocks only (an LP with free variables), and free blocks only;
- a free block whose columns are rank deficient (non-unique X_u), checking that η and the validator still converge;
- free blocks together with a large PSD block (LOBPCG path) and with dependent rows (presolve);
- warm start from a file with $S_u \neq 0$; sGS-ADMM with free blocks;
- the certificate with per-block bounds, including a missing bound (reported as unavailable);
- end to end: `PushBox_N=30_SOS` against MOSEK.

9.4 Research option: enforce the free equalities in the y-step

Instead of reaching $\mathcal{A}_u^* y = C_u$ only in the limit, the y-step can impose it exactly, as an affine constraint on the y-block. With r the non-free blocks, the y-step becomes the saddle-point system

$$\begin{bmatrix} \mathcal{A}_r \mathcal{A}_r^* & \mathcal{A}_u \\ \mathcal{A}_u^* & 0 \end{bmatrix} \begin{bmatrix} y \\ \mu \end{bmatrix} = \begin{bmatrix} \sigma^{-1} b - \mathcal{A}_r(\sigma^{-1} X_r + S_r - C_r) \\ C_u \end{bmatrix}, \qquad X_u = \sigma \mu,$$

whose matrix does not depend on σ, so it is factorized once. It is still a two-block ADMM (the y-block now carries the indicator of an affine set), but not iterate-by-iterate equivalent to (2)–(4). It needs an indefinite factorization: cuDSS in symmetric-indefinite mode, or $LDL^\top$ of the regularized quasi-definite form with $-\delta I$ in the (2,2) block, which factors without pivoting for any symmetric ordering [15]. The `NormalEquationSolver` interface reserves a saddle-point mode for it (Section 11); the option is evaluated on SOS problems after Phase 7 and is not scheduled before.

10 PSD cone projection

10.1 Design philosophy by block size

The three tiers face different constraints, and the reorganization gives them different designs rather than one abstraction stretched over all of them.

	Small ($n \le 32$)	Medium ($32 < n \le 1000$)	Large ($n > 1000$)
Typical count	hundreds to thousands	tens to thousands	a few
Goal	throughput	throughput	memory and latency per block
Parallelism	one batched call per size group	batched call per chunk, or N streams for singletons	none across blocks: each library call already fills the GPU
Dense storage	resident batch: $\sum n_b^2$ input $+\sum n_b^2$ result	same, chunked	one buffer M of $n_{\max}^2$, reused block after block
Workspace	per group (2–17 n^2 per matrix, measured)	per chunk (4.2–31 n^2 per matrix, measured) or per stream	one region W ($\approx 4.0\, n_{\max}^2$, measured) shared by all strategies
Per-block state	none	none	mode, rank, LOBPCG warm start ($n_b \times k_b$)
Strategy	fixed	fixed	chosen per iteration
Final product	strided-batched GEMM	strided-batched GEMM	SYRK on the smaller eigen-side

The tiers run concurrently on their own streams (small; medium; large) and are joined with events before the multiplier step; the large tier itself is sequential.

10.2 Interfaces

```
// Small and medium tiers: batches of equal-size blocks resident in the tier's batch buffer.
struct PsdBatch { double* data; int n; int count; std::int64_t stride; };
class BatchProjector {
 public:
  virtual ~BatchProjector() = default;
  virtual WorkspaceRequest workspace(int n, int max_batch) const = 0;
  virtual ProjectionStatus project(PsdBatch batch, ExecContext& ctx) = 0;          // in place
};

// Large tier: one block at a time through the shared arena.
struct LargeBlockJob {
  int n;                                  // this block's size (n <= n_max)
  DeviceSpan<double> M;                   // n x n dense input Z (overwritten), leading part of
    M
```

```
  DeviceSpan<double> W;                    // shared region, n_max-sized; strategies carve it up
  SvecSlice z_in, s_out;                   // the block's slices of the cone-step input and
    output
  LargeBlockState& state;                  // mode, ranks, tol_r, warm start V (n x k_alloc), D
};
class LargeBlockProjector {
 public:
  virtual ~LargeBlockProjector() = default;
  virtual std::size_t workspace_doubles(int n_max, int k_max) const = 0;          // must fit in
    W
  virtual ProjectionStatus project(LargeBlockJob& job, ExecContext& ctx) = 0;     // writes s_out
};
```

10.3 Strategy catalog

Table 4: PSD projection strategies.

Projector	Method	Precision	Use, cost and notes
`JacobiBatchedEvd`	`cusolverDnDsyevjBatched`, one call per size group	FP64	Small blocks. Tolerance 10^{-16}, 15 sweeps; `info = n+1` (tolerance not reached) is a warning.
`BatchedEvd`	`cusolverDnXsyevBatched`, one call per chunk	FP64	Medium groups with at least two blocks (CUDA $\geq$ 12.8). Accurate to machine precision up to $n = 1000$ (measured: residual $\approx 4 \times 10^{-15}$, orthogonality $\approx 10^{-13}$). Chunk size from the memory budget (B20).
`StreamedEvd`	`cusolverDnXsyevd` per block on N streams	FP64	Medium singletons, or all medium blocks with CUDA $<$ 12.8. One workspace per stream, not per matrix (B20).
`FullEvd`	`cusolverDnXsyevd` in M, then SYRK on the smaller side	FP64	Large blocks: exact phase, fallback of every large strategy, and provider of eigenvalues for `RankAnalyzer` (Section 10.5). Limited to $n \leq 32{,}768$ on CUDA 12.9 (B21).
`Lobpcg` (`Positive`)	warm-started LOBPCG [17] on Z	FP64	Large blocks whose positive rank is below ρn; $k = \lceil 1.2\, r_+ \rceil$ pairs; $O(n^2 k)$ per LOBPCG iteration.
`Lobpcg` (`Negative`)	LOBPCG on $-Z$, then $\Pi(Z) = Z + \Pi(-Z)$	FP64	Large blocks whose negative rank is below ρn (primal low rank).
`CompositeFilter`	FP64 Lanczos estimate, then the 7-step composite polynomial filter of [2]	FP16	Warm-up of large blocks with $n \geq$ `psd.large.warmup.min_size` (default 5000); 22 GEMMs on FP16 Tensor Cores with FP32 accumulation; rejected if not finite or if the norm test fails.
`FullEvdFp32` (optional)	`cusolverDnXsyevd` in FP32	FP32	Experimental; exists in the submodule, not wired into the solver.

10.4 The large-block memory model

What has to persist. Nothing of a large block's dense matrix: the cone-step input is rebuilt every iteration from svec vectors, and the result is packed back into S. Only the per-block state persists—the projection mode, the ranks and rank tolerance, and the LOBPCG warm start $V_b \in \mathbb{R}^{n_b \times k_b^{\text{alloc}}}$, $D_b \in \mathbb{R}^{k_b^{\text{alloc}}}$ with $k_b^{\text{alloc}} = \lceil 1.2\rho\, n_b \rceil$. Everything else is transient and can be shared between blocks, because they are processed one after another.

The arena. At setup the large tier reserves one region of the `WorkspaceArena`, sized for the largest large block $n_{\max}$ (Figure 2):

- M: $n_{\max}^2$ doubles, the dense input block; the EVD overwrites it with the eigenvectors;
- W: the largest workspace of any strategy at $n_{\max}$, queried from cuSOLVER at setup. On CUDA 12.9, `cusolverDnXsyevd` (FP64, vectors) needs $4.16\,n^2$ doubles at $n = 1000$ and $4.004\,n^2$ at $n = 30{,}000$ on the device, and no host workspace (measured). W also hosts, at different times, the result P of the final product (n^2), the LOBPCG buffers and the composite-filter buffers, which are all smaller than the EVD workspace;
- per block: V_b, D_b and a few scalars.

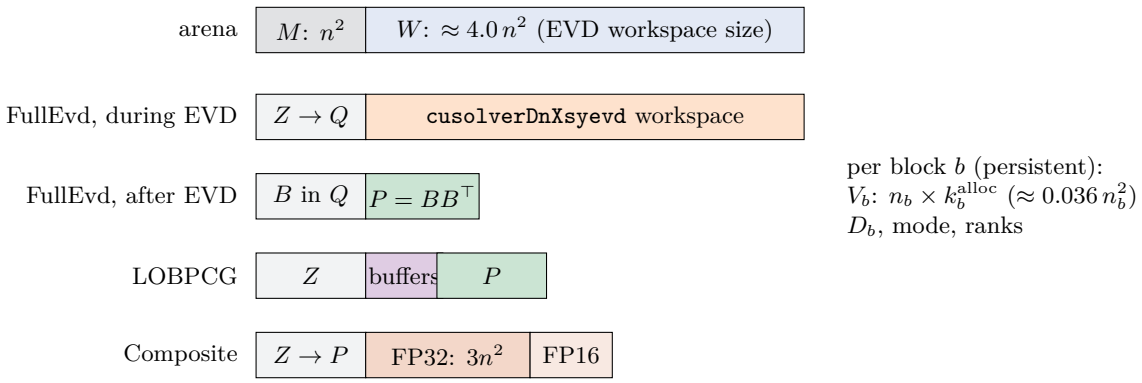


Figure 2: Large-tier arena for one block of size $n = n_{\max}$, in units of n^2 doubles. Each row shows how one strategy uses the region; the rows never coexist, because large blocks and strategies run one at a time. LOBPCG buffers: about $0.65\,n^2$ for $k = 0.036n$ with today's implementation, less after the changes of Section 10.5. Composite: $3n^2$ FP32 plus $3n^2$ FP16 values, $2.25\,n^2$ in double units, plus the Lanczos vectors.

Totals. Large-tier device memory becomes

$$\underbrace{n_{\max}^2}_{M} + \underbrace{w_{\text{evd}}(n_{\max})}_{W \,\approx\, 4.0\,n_{\max}^2} + \sum_b 1.2\rho\, n_b^2 \;\approx\; 5.0\,n_{\max}^2 + 0.036 \sum_b n_b^2 \quad \text{doubles},$$

against $\approx 7 \sum_b n_b^2$ today (B19):

Large blocks	Today	Planned	On a 48 GiB GPU
one block, $n = 20{,}000$	21.0 GiB	15.0 GiB	both fit
two blocks, $n = 20{,}000$	42.0 GiB	15.1 GiB	today leaves ≈ 6 GiB for everything else
one block, $n = 30{,}000$	47.2 GiB	33.8 GiB	today leaves almost nothing for the rest of the solver
one block, $n = 32{,}768$ (EVD limit)	≈ 56.3 GiB	≈ 40.3 GiB	only the planned layout fits

`Solver::setup` computes this plan, adds the persistent memory of the other components, prints the breakdown, and fails early with the breakdown if it exceeds the budget (`psd.large.memory_budget`). Blocks above the EVD limit are rejected at setup with a clear message (B21); supporting them is decision O5.

10.5 Large-block pipeline and algorithms

```
for each large block b, sequentially, on the large-tier stream:
    unpack  z_in(b) -> M                          # svec -> dense, O(n^2)
    pick    strategy from the large-block selection policy (below)
    project with M and W                          # result: S_b written to s_out(b)
    if rejected: unpack again (Z is not copied), then FullEvd with rank analysis
record event "large tier done"
```

FullEvd.
`cusolverDnXsyevd` in place: $M \leftarrow Q$, eigenvalues $\lambda_1 \le \cdots \le \lambda_n$. With r_+ positive and r_- negative eigenvalues: if $r_+ \le r_-$, scale the last r_+ columns of Q in place by $\sqrt{\lambda_i}$ to get B, compute the lower triangle of $P = BB^\top$ with one SYRK into W, and pack it into s_{out}; otherwise scale the first r_- columns by $\sqrt{-\lambda_i}$, compute $P' = BB^\top$, and pack $S_b = Z_b + P'$, reading Z_b from z_{in}. The final product costs $n^2 \min(r_+, r_-) \le n^3/2$ flops instead of $2n^3$ for today's GEMM, needs no n^2 temporary, and computes only the triangle that svec stores. When a rank analysis is due, the warm-start eigenvectors are copied to V_b before the in-place scaling.

LOBPCG.
Works on $M = Z$; the negative mode passes $\alpha = -1$ to its products instead of negating M twice. Its buffers come from W (B7). One product $Z[X\ R\ \Delta]$ ($n \times 3k$) per iteration serves both the residual and the Rayleigh–Ritz matrix, instead of the two products of today; the residual block is allocated $n \times k$, not $n \times 3k$. The result is formed by a SYRK on a scaled copy of the k Ritz vectors (the warm start V_b itself is not modified) and packed like FullEvd (adding Z_b in the negative mode, without today's n^2 copy of Z).

Composite filter.
Lanczos estimate on Z^2, scaling, filter in the FP32/FP16 buffers of W, result in M, then the norm test and packing (Section 10.8). On rejection the block is unpacked again and projected with FullEvd; today's n^2 copy of the input is no longer needed.

The memory model changes rounding in two places (per-block instead of strided-batched products for equal-size large blocks, and SYRK instead of GEMM), so it is introduced in Phase 6b with tolerance-based acceptance, after Phase 6a has ported the strategies bit for bit (option `psd.large.final_product = gemm_compat` keeps the old product for comparisons).

10.6 Small and medium tiers

- **Resident batches.** Each size group keeps its matrices in one batch buffer; the result buffer is separate. Today's third buffer ($Q \max(\Lambda, 0)$) is dropped by scaling the columns of Q in place by $\sqrt{\max(\lambda, 0)}$ and forming $BB^\top$ with one strided-batched GEMM (Phase 6b: rounding changes).
- **Chunked batched EVD.** The batched workspace is 4.2–31 n^2 doubles per matrix (measured; the same as one `Xsyevd` per matrix), so groups are decomposed in chunks whose size follows from the memory budget (`psd.medium.max_batch`), instead of in one call per group.
- **Streamed EVD.** One workspace per stream, sized for the largest medium singleton, instead of one per matrix: 1,000 blocks of $n = 500$ need $15 \times 9.2\,\text{MiB} \approx 137\,\text{MiB}$ instead of $\approx 8.9\,\text{GiB}$ (B20).
- **Termination measures** (eigenvalues of X, S and $C - \mathcal{A}^* y$) reuse the same machinery and workspaces; they do not keep per-matrix buffers alive for the whole solve.

10.7 Selection policy for large blocks

Per iteration and per large block, `LargeBlockPolicy` decides:

```
warm(b)  = warmup != none and n_b >= warmup.min_size      # default: composite_fp16 and 5000
k_sw     = first k > 1 with eta < warmup.switch_tol or k > warmup.switch_iter   # global, one-way
start(b) = k_sw if warm(b) else lowrank.start_iter

if warm(b) and k < k_sw:
    CompositeFilter                       -> if rejected: FullEvd
elif lowrank.enabled and k >= start(b) and (k - start(b)) % lowrank.reevaluate == 0:
    FullEvd + RankAnalyzer                -> mode = LowRankPositive | LowRankNegative | Dense
elif mode(b) is LowRankPositive or LowRankNegative:
    Lobpcg(mode(b))                       -> if rejected: FullEvd + RankAnalyzer (same k)
else:
    FullEvd
```

Defaults (Q2, Q3): blocks with $n \geq 5000$, the sizes evaluated in [2], start with the FP16 filter; large blocks with $1000 < n < 5000$ start with exact projections. The warm-up ends for all blocks at the first iteration with $\eta < 10^{-2}$ or $k > 5000$, as today. The low-rank analysis then runs every 100 iterations, from iteration 2 for blocks without a warm-up (today's CLI) and from the end of the warm-up for blocks with one (today's C++ API), so no exact EVD of a warm-up block is paid during its warm-up; $\rho = 0.03$, relaxation 1.2. Today a single flag, `switched_proj_method`, sets both the warm-up and the low-rank start for every block (B3).

Rank analysis. With the eigenvalues of the block,

$$\text{tol}_r = \max\left(10^{-8} \max(|\lambda_1|, |\lambda_n|),\ 10^{-3}\,\text{tol}\right), \qquad r_+ = \#\{\lambda_i > \text{tol}_r\}, \qquad r_- = \#\{\lambda_i < -\text{tol}_r\}.$$

Positive mode is preferred when both ranks are below ρn; the top (or bottom) k eigenpairs are kept as the warm start, with $k = \min(\lceil 1.2\, r \rceil, k^{\text{alloc}})$.

LOBPCG acceptance. A result is accepted if LOBPCG converged, all Ritz values are finite, and the k-th Ritz value is at most tol_r plus the residual norm. This is necessary, not sufficient (Ritz

values are lower bounds); the periodic exact analysis and the cone violation reported at termination bound the risk. The convergence tolerance becomes proportional to the stopping tolerance, with the constant calibrated in Phase 6a (B13).

10.8 Integrating the composite polynomial filter

1. **Code move.** The FP16 composite kernel, the Lanczos norm estimate (`approximate_two_norm`), LOBPCG and the precision-conversion and symmetrization helpers move to `src/cone/psd/kernels/`, under cuADMM's error macros (B15) and with one copy of each helper (B10). The FP32 and FP32-emulated kernels (Q3), the WIP functions (`compute_eigenpairs`, `*_auto_scale_deflate`) and the commented-out coefficient candidates stay in the frozen standalone repository; deflation is recorded as a possible future strategy.

2. **Coefficients as data.** A `constexpr` table `kCompositeFp16[7][3]` holds the refined half-precision coefficients $\tilde{f}$ of [2, Table 2] (identical to the code today), obtained by Remez initialization and gradient refinement, with a provenance comment; the Polar Express coefficients [16] are kept only as a test baseline. Each step evaluates $aX + bX^3 + cX^5$ with three GEMMs (X^2, X^3, X^3X^2).

3. **Algorithm per block.** Lanczos (20 steps) on Z^2 gives θ and q; the estimate $u = \sqrt{\theta + \|Z^2q - \theta q\|}$ [2, Theorem 2] is not a certified bound when the Ritz value is not closest to $\lambda_{\max}(Z^2)$, so it is scaled by a safety margin $\mu = 1.05$: $s = \mu u$, $X_0 = Z/s$, $X_t = \tilde{f}_t(X_{t-1}/d_t)$ for $t = 1, \ldots, 7$ with damping $d_t = 1.01$, result $P = s \cdot \frac{1}{2}X_0(I + X_T)$, symmetrized. The damping location (input, as coded, versus output, as described in [2, §3.4]) is documented (B11).

4. **Precision and math modes (B1).** The GEMMs run on FP16 Tensor Cores with FP32 accumulation and output (`cublasGemmEx`, compute type `CUBLAS_COMPUTE_32F_FAST_16F` set in each call); the polynomial updates run in FP32 and the Lanczos estimate in FP64. The handle uses `CUBLAS_DEFAULT_MATH`; `CUBLAS_TENSOR_OP_MATH` is never set. The FP16 output is bitwise identical under both modes (measured at $n = 1000$, 2000, 5000), so the B1 fix does not change it. Why FP16 only: the filter runs only while $\eta \geq 10^{-2}$, where an error of about 10^{-3} does not limit progress; on H100/H200 (SXM) plain FP32 GEMMs peak at the FP64 Tensor Core rate (about 67 TFLOPS, against about 990 for dense FP16); and the FP32-emulated variant needs Blackwell GPUs that are not available for validation. A TF32 variant can be added later as a new option value.

5. **Safety checks (B22, moved to Phase 1).** Reject and fall back to FullEvd if the result is not finite or if $\|P\|_F > (1 + 10^{-2})\|Z\|_F$: a free necessary condition, since the projection is nonexpansive with $\Pi(0) = 0$. It also catches finite but wrong results from an underestimated scale.

6. **Determinism (B8).** The Lanczos start vector comes from a counter-based generator seeded by `(options.seed, block index)`.

7. **Workspace (B7).** The FP32/FP16 buffers and the Lanczos vectors live in W (Figure 2); nothing is allocated per call.

8. **Exposure (B4).** Options `psd.large.warmup` $\in$ {`none`, `composite_fp16`} and `psd.large.warmup.min_size`, in the CLI (`--psd-warmup`, `--psd-warmup-min-size`) and MATLAB.

9. **Scope.** Large blocks with $n \geq$ `min_size`: the GEMM advantage needs n in the thousands, and $n = 5000$ is the smallest size evaluated in [2]. Phase 6a benchmarks the filter against

`FullEvd` at $n \in \{2000, 3000, 5000, 10000, 20000\}$ on the pinned GPUs to confirm or move the threshold. A strided-batched filter for medium groups is a benchmark item, not a default.

10.9 PSD options

Option	Default	Today
`psd.small_max`, `psd.medium_max`	32, 1000	`SMALL_MAT_LIMIT`, `MEDIUM_MAT_LIMIT` macros
`psd.medium_batch_min`	2	public field `medium_batch_min_count`
`psd.medium.max_batch`	from the budget	whole group in one call
`psd.medium_streams`	15 (must be ≥ 1)	`--streams`; 0 accepted by the MEX (B17)
`psd.jacobi_tol`, `psd.jacobi_max_sweeps`	10^{-16}, 15	`BatchEigParameter` defaults
`psd.large.memory_budget`	0.9 of free memory	none (allocation fails)
`psd.large.final_product`	`syrk` (`gemm_compat` in 6a)	GEMM with an n^2 temporary
`psd.large.warmup`	`composite_fp16` from 6a, `none` before	CLI/MEX `EIG_FP64`; C++ API `COMPOSITE_FP16` on every large block
`psd.large.warmup.min_size`	5000	none (every large block)
`psd.large.warmup.switch_tol`, `.switch_iter`	10^{-2}, 5000	10^{-2}; CLI 0 / MEX and API 5000
`psd.lowrank.enabled`	true	`--lobpcg`
`psd.lowrank.start_iter`, `.reevaluate`	2 (warm-up blocks: end of the warm-up), 100	coupled to the warm-up switch for every block; 100
`psd.lowrank.ratio`, `.relaxation`	0.03, 1.2	`LOBPCG_*` macros
`psd.lowrank.max_iter`, `.tol`	20, relative	20, absolute 10^{-5}
`psd.lowrank.rank_rtol`, `.rank_stop_factor`	10^{-8}, 10^{-3}	macros
`psd.composite.margin`	1.05	`COMPOSITE_FP16_SCALE_MARGIN` macro
`psd.composite.lanczos_steps`, `.norm_slack`	20, 10^{-2}	20; no norm test

11 Linear systems

11.1 Role

Every y-step solves $(\mathcal{A}\mathcal{A}^* + \varepsilon I)\, y = r$ with the scaled, row-normalized $\mathcal{A}$; ADMM needs one solve per iteration and sGS-ADMM two. The matrix does not depend on σ, so a direct method factors it once at setup, and the per-iteration *solve* dominates the cost: SPOT runs are bound by CPU CHOLMOD solves of 5–93 ms each [19]. Today CHOLMOD [13] always factors on the CPU (supernodal, falling back to simplicial $LDL^\top$); solves run on the CPU with a device synchronization and two PCIe copies per iteration, unless the factor is dense ($m \geq 2000$, $m^2 \leq$ `INT_MAX`, $8m^2$ bytes $\leq 25\%$ of free memory, $\mathrm{nnz}(L) > 0.3\, m(m+1)/2$, simplicial $LDL^\top$ factor), in which case L and D are copied to the GPU and each solve is two `cublasDtrsv` calls.

11.2 Supported methods

Method	Backend	Factor / solve	Status
sparse direct	`CholmodSolver`	CPU / CPU	today's default; ported bitwise in Phase 7
sparse direct	`CudssSolver`	GPU / GPU	Phase 7; optional build; intended default once the benchmark confirms it
dense direct	`CholmodSolver`, dense mode	CPU / GPU `trsv`	today, for dense factors
dense direct	`DenseCusolverSolver`	GPU / GPU	Phase 7: `Xpotrf`/`Xpotrs`, `Dsytrf`/`Xsytrs`, optional `Xtrtri` inverse factor
iterative	`PcgSolver` + preconditioner	— / GPU	reserved: interface in Phase 7, solver on the research track
saddle point (free variables)	cuDSS indefinite, quasi-definite $LDL^\top$	GPU or CPU	reserved (Section 9)

11.3 Interface

The interface is written for both exact and inexact solves, so that iterative methods can be added without touching the algorithm layer:

```
struct SolveRequest {
  double rel_tol = 0.0;           // 0: exact (direct backends); > 0: inexact solve allowed
  int max_iter = 0;               // iterative backends only
  bool use_initial_guess = false; // warm start from the content of y (e.g. the previous y-step)
};
struct SolveReport { int iterations = 0; double rel_residual = 0.0; bool converged = true; };

struct Capabilities { bool exact; bool device_solves; bool saddle_point; bool sigma_dependent_ok;
     };

class NormalEquationSolver {
 public:
  virtual ~NormalEquationSolver() = default;
  virtual std::string_view name() const = 0;
  virtual Capabilities capabilities() const = 0;
  virtual SymbolicInfo analyze(const NormalSystem& sys) = 0;     // ordering, nnz(L), memory
  virtual FactorInfo factorize(const NormalSystem& sys) = 0;     // no-op for matrix-free solvers
  virtual void update(const NormalSystem& sys) {}                // future sigma-dependent systems
  virtual SolveReport solve(DeviceSpan<const double> rhs, DeviceSpan<double> y,
                            const SolveRequest& req, ExecContext& ctx) = 0;
};

class NormalOperator {           // matrix-free: v -> (A A^T + eps I) v with two SpMVs (no A A^T
    formed)
 public:
  void apply(DeviceSpan<const double> v, DeviceSpan<double> out, ExecContext& ctx) const;
};
class Preconditioner {
 public:
  virtual ~Preconditioner() = default;
  virtual void setup(const NormalSystem& sys, ExecContext& ctx) = 0;
  virtual void apply(DeviceSpan<const double> r, DeviceSpan<double> z, ExecContext& ctx) const =
    0;
};
```

`NormalSystem` describes $\mathcal{A}\mathcal{A}^* + \varepsilon I$ (assembled once, lower triangle, with `cusparseSpGEMM` on the GPU or `cholmod_aat` on the host), optionally the saddle-point form of Section 9, and the matrix-free operator. Today's system does not depend on σ; `update()` exists for future variants whose system does (proximal terms, quadratic objectives), where a penalty rule with finitely many changes also bounds the number of refactorizations.

11.4 Backends

`CholmodSolver` (always built). Today's behavior moved out of `SDPSolver`: supernodal with simplicial $LDL^\top$ fallback; CPU solves through pinned host buffers and asynchronous copies on the solver stream; the dense mode keeps all of today's conditions, so the CHOLMOD path stays bitwise.

`CudssSolver` (optional, `CUADMM_WITH_CUDSS`). NVIDIA cuDSS [14], distributed separately from the CUDA toolkit and NVIDIA's replacement for the sparse Cholesky routines of cuSOLVER, which the CUDA 12.9 headers mark deprecated in favor of cuDSS. Analysis, factorization and solve on the device; SPD (Cholesky) or symmetric indefinite ($LDL^\top$) for rank-deficient and saddle-point systems; no host synchronization per solve. The minimum version is pinned, availability is checked at runtime, and its deterministic mode is enabled under `determinism = strict` (it is off by default in cuDSS).

`DenseCusolverSolver` (optional). `cusolverDnXpotrf/Xpotrs` for SPD, `Dsytrf/Xsytrs` otherwise, or an explicit inverse factor from `Xtrtri` so each solve is two triangular matrix–vector products, which parallelize better than `trsv`. Useful when m is a few thousand and the factor is dense. The reference GPUs have weak FP64 (about 1/64 of FP32), which penalizes dense FP64 factorizations.

11.5 Selection

1. If `linsys.backend` is not `auto`, use it.
2. Otherwise run the symbolic analysis (`cholmod_analyze`, or the cuDSS analysis phase once cuDSS is admitted to `auto`) to get $\mathrm{nnz}(L)$ and the memory need.
3. Until the benchmark of Section 11.8 reports, `auto` chooses CHOLMOD with today's rule for the dense mode ($m \geq 2000$, $m^2 \leq$ `INT_MAX`, $8m^2$ bytes within the budget, $\mathrm{nnz}(L) \geq 0.3\, m(m+1)/2$, simplicial factor). After the benchmark, `auto` prefers cuDSS when its factor fits the budget.
4. With `linsys.autotune = true`, factor with the two best candidates, time ten solves each and keep the faster. The choice and the timings go to the log and the summary.

Throughout the bitwise phases the backend is CHOLMOD and presolve is off.

11.6 Inexact and preconditioned solves (reserved)

Direct factorizations stop scaling when the factor does not fit or its fill makes each solve expensive. The interface above lets an iterative backend take over without changes elsewhere:

- **Solver.** Preconditioned CG on the matrix-free `NormalOperator`, warm-started from the previous y (successive y-steps change little near convergence), with $O(m)$ vectors of memory.

- **Accuracy schedule.** Inexact y-steps are admissible when the errors are summable, as in the inexact sGS-ADMM of [6]: $\|(\mathcal{A}\mathcal{A}^* + \varepsilon I)\hat{y} - r\| \le \epsilon_k$ with $\sum_k \epsilon_k < \infty$, for example $\epsilon_k = \min(\kappa\,\eta_{k-1},\ \epsilon_0 k^{-1.1})$. The policy that produces ϵ_k is a small strategy object of its own; `SolveReport` feeds the iteration count and residual to the observers.
- **Preconditioners**, behind the `Preconditioner` interface: Jacobi (weak here, since the row normalization makes the diagonal of $\mathcal{A}\mathcal{A}^*$ close to 1 for rows with $\|A_i\| \ge 1$); block-Jacobi over groups of constraints; incomplete Cholesky (in-house or third party: cuSPARSE's `csric02` is marked deprecated in CUDA 12.9); the exact factor of a nearby matrix, e.g. from the previous problem in a sequence of related trajectory-optimization solves; and mixed precision (an FP32 factor with FP64 iterative refinement).
- **Scope.** Inexact solves change the iterates, so they are never used in the bitwise phases. The interface and `NormalOperator` land in Phase 7; `PcgSolver` and the preconditioners are a research item with their own benchmark.

11.7 Rank deficiency and presolve

Moment relaxations often have dependent constraints (the SPOT data); a tiny regularization $\varepsilon = 10^{-15}$ plus simplicial $LDL^\top$ makes them work today, and it is fragile (B9). A `Presolve` step detects dependent rows from the factorization pivots (or a sparse QR), removes them after checking that b is consistent, and maps y back at the end. Regularization stays as a fallback option. Presolve changes b, hence b_s and the meaning of a given σ, so it is off during the bitwise phases and its effect is measured in Phase 7. cuDSS in SPD mode also needs a nonsingular matrix.

11.8 Benchmark protocol

Run early, before cuDSS can become a default. Problems: SPOT (PushBox, PushT, PlanarHand at $N = 1, 10, 30$, including the SOS form), pendulum (`licols`), and PLATO / Mittelmann instances including one with a dense factor (`neosfrbr25`). Metrics: analysis, factorization and per-solve time; peak memory; relative residual $\|\mathcal{A}\mathcal{A}^* y - r\|/\|r\|$; end-to-end time to $\eta < 10^{-4}$. GPUs: the RTX 6000 Ada workstation and an H100/H200 node.

12 Problem input and output

Today. The solver reads one format: a directory of text files (`blk.txt`; `At.txt` with 0-based `row col value` triplets of $\mathcal{A}^*$ in svec form; `b.txt`; `C.txt`; `con_num.txt`). Every other format is converted outside the solver, by MATLAB scripts (`examples/sdpt3_to_txt.m`, `sedumi_to_txt.m`, `mosek_to_txt.m`) or by the Python SDPA converter of the PLATO experiments; the MEX takes the same arrays in memory. The text reader parses about 3.5×10^6 nonzeros per second: 20 million nonzeros take 708 MB of text and 5.7 s, against 320 MB and 0.16 s for the same data in binary read from the page cache (measured on the reference workstation). It also builds COO triplets before CSC, doubling the host memory, and stores sizes in `int` (B27). At 10^9 nonzeros that would be about 35 GB of text and five minutes of parsing at every run.

Design. Every reader produces the same `ProblemData`; all conventions (objective sense, svec layout, block mapping) live in `src/io/` and nowhere else. Sizes are 64-bit in the readers and in

the native format; setup checks that the problem fits the solver's current 32-bit indexing and says so if it does not. Readers fill the CSC arrays directly, without a COO copy. Each reader prints a conversion report (what was mapped, mirrored, merged or simplified), which also goes into the summary.

Format	Typical source	Support
cuADMM HDF5 (`.h5`)	large problems, converted once	native: read and write; chunked, 64-bit, optional compression
SDPA sparse (`.dat-s`, `.dat-s.gz`)	SDPLIB, DIMACS, Mittelmann (PLATO), Kocvara	C++ reader, with the checks of the PLATO converter
SeDuMi `.mat` (`A` or `At`, `b`, `c`, `K`)	PLATO, DIMACS, SOSTOOLS, GloptiPoly	C++ reader (matio); MEX and Python take the same variables
SDPT3 `.mat` (`blk`, `At`, `C`, `b`)	SDPT3 and SDPNAL+ users	same
MOSEK MATLAB `prob` (`.mat`)	SPOT; MOSEK toolbox users	same; the objective sense is an input
legacy TXT directory	today's data and tests	read only; deprecated
CBF (`.cbf`)	CBLIB	with second-order cones (Phase 9)

Mappings.

SDPA.
SDPA's dual (D) is cuADMM's primal: $C = -F_0$, $A_i = F_i$, $b_i = c_i$, and the returned y is $-x_{\text{SDPA}}$. Negative block sizes are diagonal blocks (`l`). Lower-triangle entries are mirrored; duplicate entries, non-finite values and indices out of range reject the file; explicit zeros are dropped; the DIMACS block kinds are kept for the error measures.

SeDuMi.
`K.f` $\to$ `u`, `K.l` $\to$ `l`, `K.s` $\to$ `s`; `K.q` and `K.r` with Phase 9; complex data (`K.scomplex`, `K.xcomplex`) is rejected. `A` ($m \times N$ or $N \times m$) and `At` ($N \times m$) are both accepted; when $N = m$ the name decides the orientation. PSD blocks are full column-major n^2 vectors and need not be symmetric: the svec entry of (k, l), $k < l$, is $(A_{kl} + A_{lk})/\sqrt{2}$. PLATO's `.mat` files are in this format (with `At`).

SDPT3.
`blk{p,1}` $\in$ {`s`, `l`, `u`} (`q` with Phase 9). An `s` entry that lists several sizes is split into separate blocks. `At{p}` is already in svec form with the same convention as cuADMM; `C{p}` is converted.

MOSEK `prob`.
Bar variables (`bardim`) become `s` blocks. `bara` and `barc` list the lower triangle (subk $\geq$ subl) of symmetric matrices, so the svec value is `val` on the diagonal and $\sqrt{2}\,$val off it. Scalar variables (columns of `a`): missing or infinite bounds $\to$ `u`, $[0, \infty) \to$ `l`, other bounds by a shift and, for a finite upper bound, a slack. Rows with `blc` = `buc` are equalities; other rows get a slack in an `l` block. `cfix` is the objective offset. `cones` and `accs`/`f`/`g` come with Phase 9; integer variables are rejected. The struct does not store the objective sense, which is an argument of `mosekopt`: the reader takes it from a variable `sense` saved next to `prob`, or from `io.sense`, and stops if neither is given, because SPOT's SOS files are maximization problems and a silent default would solve a different problem.

Exact simplifications at read time (option `io.simplify`, on by default, each one reported). A 1×1 PSD block becomes an `l` block. A PSD block whose data ($\mathcal{A}$ and C) touch only its diagonal becomes an `l` block of the same size; this is exact, because such an $X \succeq 0$ can be replaced by $\mathrm{diag}(X) \geq 0$ without changing the optimal value, and the dual slack is diagonal. It matters for PLATO's SeDuMi files: `buck3.mat` stores the 544-entry diagonal block of `buck3.dat-s` as a 544×544 PSD block, which would cost an $n = 544$ eigendecomposition per iteration instead of 544 scalar operations; read with this simplification, `buck3.mat` gives the same data as `buck3.dat-s` to within rounding (checked). Solutions are mapped back to the file's block structure on output. The legacy TXT reader reads files exactly as today and never simplifies, so results on existing TXT data do not change.

Output. Solutions are written in the convention of the input: `.h5` (group `/solution`); SDPA ($x = -y$, X, Y); SeDuMi (`x`, `y`, `z` as full vectors); SDPT3 (cell arrays); MOSEK (the `res.sol.itr` fields `barx` and `bars` as lower triangles, `xx`, `slx`, `y`), so that SPOT's `recover_mosek_sol_blk` works unchanged. Objective values are reported in the file's sense, including the offset.

Native format. One HDF5 file, version 1:

```
/                  attrs: format = "cuadmm", version = 1, sense = "min" | "max",
                          objective_offset, source, source_sha256,
                          svec = "upper, column-major, off-diagonal x sqrt(2)"
/blocks/kind       uint8 (B)       's', 'l', 'u' ('q' later)
/blocks/size       int64 (B)
/At/colptr         int64 (m+1)     CSC of A^T: one column per constraint
/At/rowind         int64 (nnz)     svec index, 0-based, sorted within each column
/At/val            float64 (nnz)
/b/index, /b/val   int64, float64  sparse
/C/index, /C/val   int64, float64  sparse
/meta/...          optional: DIMACS block kinds, conversion report
/start/...         optional warm start: X, y, S, sigma
/solution/...      written by the solver: X, y, S, status, residuals, objective values
```

Datasets are chunked (about 10^6 entries per chunk) and optionally compressed; the reader copies chunks straight into the CSC arrays. MATLAB writes the file with `h5create`/`h5write` and Python with `h5py`; the tool `cuadmm-convert` converts any supported format into it once, for problems too large to parse at every run.

Dependencies and detection. HDF5 and matio (BSD-style licenses), and zlib for `.gz`. Build options `CUADMM_WITH_HDF5` and `CUADMM_WITH_MATIO` are on when the libraries are found; without matio, the `.mat` formats are read by the MATLAB and Python front-ends, which pass the variables in memory. The CLI lists the formats compiled in. The format is detected from the extension and, for `.mat`, from the variables present (`prob` → MOSEK; `blk` with `At`, `C`, `b` → SDPT3; `K` with `A` or `At` → SeDuMi), and can be forced with `io.format`.

SPOT. SPOT builds the MOSEK `prob` struct from coordinate lists produced by its C++ core (`mosek_standard_sdp_test_1.m` and `_2.m`). Moment relaxations are minimizations over bar variables with `barc`; SOS relaxations are maximizations with free scalar variables and no `barc`. The sample files in `examples/SPOT/data/MOSEK` are MATLAB v5 files that contain only `prob`; read with the mapping above, three of them (PushBox $N = 30$ moment and SOS, PushT $N = 10$ moment)

reproduce their TXT conversions in `examples/SPOT/data/TXT` to within rounding (checked). Two changes on the SPOT side make the hand-over exact and fast (O9): save `sense` next to `prob`, and, for large instances, write the cuADMM HDF5 file directly from the same coordinate lists.

13 Scaling

A `Scaling` object holds ν, b_s, C_s and provides `scale_problem`, `scale_iterate`, `unscale_iterate`, `to_original_residuals` and `sigma_to_original`. It is the only code that knows the scaling; the warm start, the termination, the validator and the result use it. Option `scaling.enabled` (default true) allows running unscaled for diagnostics. Two known limitations are recorded, not changed: the residual vote is not invariant under rescaling b and C, because of the "1+" in its normalizers [11, 18], and σ_0 is given in scaled units. The unused fields `bscale2`, `Cscale2` are deleted.

14 Core infrastructure

- **`GpuContext`**: device, library handles per stream role, cuDSS handle, and the math mode of each handle set explicitly at creation (B1).
- **`StreamPool` and `ExecContext`**: named streams (main, small, medium[N], large, linsys, copy). An `ExecContext` bundles a stream, its handles and a workspace slice; cross-stream dependencies use events. Everything runs on non-blocking streams owned by the pool, which removes the class of bugs behind B2. A `single_stream` mode runs everything on one stream; it must reproduce the concurrent run bit for bit, because `compute-sanitizer racecheck` cannot see B2-type races between streams in global memory.
- **`WorkspaceArena`**: components declare `WorkspaceRequest`s at setup; the arena allocates once, hands out aligned slices, and lets components that never run at the same time share a region (the large tier, Section 10.4). No `cudaMalloc`/`cudaFree` inside the loop.
- **Buffers**: `DeviceBuffer<T>`, `PinnedHostBuffer<T>`, `DeviceSpan<T>` with `std::size_t` sizes; the overflow checks of `BlockStructure` are kept.
- **Checks**: one set of error macros (B15), a launch check after every kernel (B16), exceptions instead of `exit()`.
- **Determinism**: option `determinism` $\in$ {`strict` (default), `fast`}.
- **Logging**: a `Logger` with levels; library code does not call `printf`.

15 Unified options

Table 5 is the single table of defaults (PSD options are in Section 10). The CLI flag names are derived from it (`algorithm.tau` becomes `--tau`, with today's flags kept as aliases for one release) and the MATLAB options struct uses the same nested field names.

Table 5: Solver options and defaults ("O" marks an open decision of Section 18).

Option	Default	Notes
`algorithm.scheme`	`admm`	`admm` or `sgs_admm`; replaces `switch_admm` and the compile flag.
`algorithm.tau`	1.618	Validated in $(0, 2)$; 1.918 available; avoid $\tau \geq 1.99$ from cold starts (Q4).
`algorithm.scheme_switch_iteration`	0 (off)	Hybrid mode until Q5 is decided.
`algorithm.max_iter`, `.time_limit`	10^6, none	As today in the CLI.
`algorithm.tol`	10^{-4}	Internal stopping tolerance on η.
`penalty.policy`	`balance_freeze` from 4b; before: `residual_balancing` (O1)	`fixed`, `legacy_adaptive`, `residual_balancing`, `balance_freeze`.
`penalty.sigma0`	100	Scaled units; clamped into the bounds except in `legacy_adaptive`.
`penalty.sigma_min`, `.sigma_max`	10^{-3}, 10^3	`legacy_adaptive`: 10^{-6}, 10^6.
`penalty.factor`, `.step_schedule`	2, `constant`	Table 2; today's working tree has 1.2.
`penalty.win_ratio`, `.guard`	1.35, true	The win-count hysteresis (not today's `ratioconst`).
`penalty.residual_weight`	1	Today's `ratioconst`, a weight on η_p.
`penalty.schedule`	as in §3.5	List of (up to iteration, period) pairs.
`penalty.freeze_iteration`	off / 10^4	Off for `residual_balancing`, 10^4 for `balance_freeze`.
`penalty.pingpong_window`, `.pingpong_net`, `.pingpong_min_iter`	8, 2, 1000	`balance_freeze` only.
`penalty.freeze_value`, `.freeze_bias`	band centre, 1	`balance_freeze` only.
`scaling.enabled`	true	
`linsys.backend`	`auto`	`auto`, `cholmod`, `cudss`, `dense`; `auto` admits cuDSS after the benchmark.
`linsys.regularization`	10^{-15}	Added to the diagonal of $\mathcal{A}\mathcal{A}^*$.
`linsys.presolve`	true from Phase 7	Off in the bitwise phases.
`linsys.memory_fraction`, `.autotune`	0.25, false	Budget for dense or cuDSS factors.
`linsys.inexact`, `.preconditioner`	off, `none`	Reserved (Section 11.6).
`certificate.block_bounds`	none	Per-block bounds R_b; required for a certified bound with free blocks.
`certificate.trace_bound`, `.trace_bound_scale`	none	Today's `--trace-bound`, `--trace-bound-scale`.
`validation.interval`, `.tol`, `.threads`	0, `algorithm.tol`, 1	Validation-aware stopping as today.
`validation.strict_dimacs_tol`	0 (off)	Today's `--strict-dimacs-tol`.
`validation.checkpoint_iterations`, `.checkpoint_dir`, `.final_save_dir`	none	Today's checkpoint options.
`io.format`	`auto`	`h5`, `sdpa`, `sedumi`, `sdpt3`, `mosek`, `txt` (Section 12).
`io.sense`	from the file	Required for MOSEK files without a `sense` variable.
`io.simplify`	true	1×1 and diagonal-only PSD blocks become `l` blocks (reported); never applied to legacy TXT input.
`io.warm_start_dir`, `io.save_dir`, `io.save_format`	none, none, input's	Today's `--warm-start-dir`, `--save-solution`.

Option	Default	Notes
`output.verbosity`, `.history`, `.summary`, `.sigma_log`	1, off, off, off	Observers; 17 significant digits.
`seed`, `determinism`	0, `strict`	
`gpu.device`	0	One GPU per solver.

Defaults that change. The facade adopts the CLI's defaults, which differ from today's `SDPSolver` defaults: σ_0 from 1.0 to 100 (`solver.h:517`; its comment says `2e2`); the large-block warm-up from `COMPOSITE_FP16` on every large block (`solver.h:512`) to `none` until Phase 6a and `composite_fp16` on blocks with $n \geq 5000$ from then on; for blocks without a warm-up, the low-rank start from the warm-up switch at 5000 to iteration 2; and the validation tolerance from the API value to `algorithm.tol`. From Phase 6a, CLI and MATLAB users also get the FP16 warm-up on blocks with $n \geq 5000$, which today they cannot select (B4). Phase 7 then turns presolve on and, after the benchmark, may make cuDSS the backend. Each change is listed in the release notes.

16 Testing, benchmarking and continuous integration

- **Unit tests** per component: cone properties (idempotence, Moreau decomposition, orthogonality, nonexpansiveness, membership) for every cone, the free cone included; every PSD projector against an FP64 reference on synthetic spectra (full rank, low rank $\pm$, clustered, tiny n, $n = 1000$, $n > 1000$), with tolerances per precision (FP16 composite below 2×10^{-3} median relative error, exact projectors near machine precision); SYRK versus GEMM products on both eigen-sides; normal-equation backends on full-rank and rank-deficient matrices; penalty policies on scripted residual sequences at full precision; option parsing, validation and unknown names.
- **Memory tests**: the setup-time memory plan against the measured peak, for one and several large blocks, and a block at the EVD limit on a large-memory GPU.
- **Integration tests**: generated problems with known optimum for every cone (free blocks as in Section 9) and every projector path, both schemes, $\tau \in \{1.618, 1.918\}$, both direct backends.
- **Golden histories**: per-iteration $(\text{pobj}, \text{dobj}, \eta_p, \eta_d, \sigma)$ of a fixed set of problems, per policy, regenerated only by the published generator script when a change is intended.
- **Formats**: each reader on hand-made edge cases (lower-triangle and duplicate entries, block groups, bounds and ranges in MOSEK data, missing sense, $N = m$ in SeDuMi); instances available in several formats (PLATO SDPA and SeDuMi, SPOT MOSEK and TXT) give the same `ProblemData` up to the reported simplifications; HDF5 round trips; solution output read back by SPOT's `recover_mosek_sol_blk`.
- **End to end**: a small SPOT (including SOS) and PLATO subset checked against MOSEK.
- **Benchmarks** (`bench/`): projector micro-benchmarks, normal-equation backends, end-to-end time; per-iteration time must stay within about 2% on pendulum $N = 80$ and three PLATO problems (run-to-run noise is about 0.6% [19]).
- **CI**: build matrix with and without cuDSS; CUDA 12.8 and 12.9; explicit CUDA architectures (today `CUDA_ARCHITECTURES OFF`); nightly `compute-sanitizer` (memcheck, racecheck, sync-

check) and the single-stream bitwise check; GPU jobs through Slurm scripts until CI has GPU runners.

17 Migration plan

Each step is one or a few pull requests on a feature branch, merged in order through pull requests (the `main` ruleset requires them). "Bitwise" means identical per-iteration histories on the golden set and on two real problems, on each pinned machine and configuration (GPU model and SM count, driver, CUDA, SuiteSparse and BLAS versions, thread count, CUDA architectures).

Pre-0 — Names and housekeeping.
In today's `SDPSolver`: restore `legacy_adaptive` to `9bb4ba5` bit for bit (no guard, $[10^{-6}, 10^{6}]$, `Monitor1`); add `residual_balancing` with the October 13 rule and the options `penalty.factor` (default 2; the working-tree 1.2 becomes an option value); make `residual_balancing` the default until Phase 4b (O1); write the policy, its parameters and the build commit into the summary JSON; make unknown policy names an error; fix the CLI help and README (B18). Housekeeping: drop the LaTeX `.synctex.gz` from the tree; the experiment authors publish the golden generator, push and tag the cited commits, and fix the dangling references (B26). *Acceptance:* the legacy golden test passes again (B14); `residual_balancing` goldens are generated by the published script.

Phase 0 — Safety net.
Fixed-σ baselines first: they do not depend on the penalty policy, and `main` must reproduce the PLATO fixed-σ histories bit for bit (`trto1`, `mater-1`; under 0.1 H200-hour). One baseline per pinned machine and configuration; GPU gate scripts; the CHOLMOD rank-deficient test made independent of the SuiteSparse version (B9). *Acceptance:* baselines stored; CI green.

Phase 1 — Cleanup and one-line fixes.
Delete dead code (P4); remove `CUADMM_PLAIN_ADMM_ONLY` (B5); fix B1, B2 (event before the copy), B6, B12, B16 (launch checks; `exit()` to exceptions), B17 (validate MEX scalars), B21 (EVD size check at setup), B22 (norm test). *Acceptance:* bitwise on the exact paths; the composite accuracy test tightened to FP32 levels.

Phase 2 — Options and front-ends.
`SolverOptions`, the `Solver` facade over the existing class, CLI and MATLAB rebuilt on it (`cuadmm.solve(At, b, C, K, opts)` with a positional shim for one release); B3 and B4 resolved with the defaults of Section 10.7. *Acceptance:* bitwise for equivalent options; option table generated into the documentation.

Phase 2b — Input and output formats.
After Phase 2, in parallel with Phases 3–4: the readers, writers and HDF5 schema of Section 12, `cuadmm-convert`, and B27–B29. *Acceptance:* the TXT reader gives the same `ProblemData` as today; the format tests of Section 16 pass; PLATO instances read from either format reach their published objectives; reading 10^8 nonzeros from `.h5` is limited by disk bandwidth, not by parsing.

Phase 3 — Thin core.
`GpuContext`, `ExecContext`, buffer and span types, unified checks (B15). The stream pool and the arena come with Phase 6b. *Acceptance:* bitwise.

Phase 4 — Algorithm layer.
`AdmmState`, `IterationScheme`, the penalty policies of Section 7.3 (including `balance_freeze`, implemented but not yet the default), `KktEvaluator`, `StoppingRule`, `BestIterateTracker` (B23), observers with post-step residuals and 17-digit histories (B25). *Acceptance:* bitwise for ADMM with each existing policy; for sGS-ADMM: with `FixedPenalty` and $\tau = 1.95$, it matches a tagged build of today's sGS path built without `CUADMM_PLAIN_ADMM_ONLY`, at fixed σ and stopping tolerance 0 (so that the hidden τ reduction never fires).

Phase 4b — Tune and adopt `balance_freeze`.
The default policy must have finite total σ variation, which `balance_freeze` has by construction; this step chooses its parameters. CPU screen on the validated replica [18] (15–20 core-hours): `balance_freeze` over Table 2 in a fractional design (factor, step schedule and K_f first, then the freeze value and bias for the two best), each at $\sigma_0 \in \{0.5, 1, 2\}$ on the nine small PLATO problems. Then GPU gates (at most 12.6 H200-hours): G0 fixed-σ reproduction; G1 `trto1`, `buck1` (600 s caps), `vibra1`, `mater-1`, `mater-2`, `shmup1` (1,800 s); G2 pendulum $N = 80$ at $\sigma_0 \in \{1, 100\}$ against 124 s (old rule) and 625 s (fixed $\sigma = 1$); G3 `mater-3` to `mater-6`. *Gates:* no problem lost that fixed $\sigma = 1$ or the old rule solves within the cap; the logged $\sum |\log_2 \Delta\sigma|$ stays bounded and the changes stop; no change followed by a $\geq 10\times$ rise in the validated error; a better shifted geometric mean of time (shift 10 s) than both `residual_balancing` and fixed $\sigma = 1$; robust across σ_0. *Outcome:* the best passing setting becomes the default of `penalty.policy`. If no setting passes all gates, adopt the one that loses the fewest problems, document the losses, and revisit the freeze design (O2, O8).

Phase 5 — Cone layer.
`Cone`, `ConeProduct`, `ConeRegistry`; `FreeCone`, `NonnegativeCone`, and the PSD cone wrapping the current tiers unchanged; the certificate through `Cone::measure` with per-block bounds (B24); the validator kept independent with host reference measures; the free-variable tests of Section 9. *Acceptance:* bitwise; new tests pass.

Phase 6a — PSD strategies, bit for bit.
The projectors of Table 4 on today's memory layout, `LargeBlockPolicy`, `RankAnalyzer`; the FP16 composite code moved in (B8, B10, B11, B13); the submodule removed only after the parity tests pass. Once B8 makes the filter deterministic, the default warm-up (`composite_fp16` for $n \geq 5000$) is switched on, after the threshold benchmark of Section 10.8. *Acceptance:* bitwise on the exact and LOBPCG paths (`gemm_compat`); for the composite path, whose Lanczos start is random today (B8), accuracy, fallback and end-to-end tests with and without the warm-up.

Phase 6b — PSD memory model.
Stream pool and arena; large-block arena and per-block pipeline (Sections 10.4–10.5); SYRK products; LOBPCG and Lanczos without per-call allocation (B7); small and medium workspace fixes (B19, B20). *Acceptance:* within rounding (relative differences in the histories at the level of the projection error); measured peak memory within 5% of the setup plan; time per iteration not worse than Phase 6a.

Phase 7 — Linear systems.
The interface types (`NormalSystem`, `NormalOperator`, and `NormalEquationSolver` with the inexact hooks); the backends `CholmodSolver` (bitwise), `CudssSolver` and `DenseCusolverSolver`; presolve; the benchmark report. *Acceptance:* CHOLMOD path bitwise; cuDSS within 10^{-10} relative residual per solve and the same iteration counts within a few percent end to end.

Phase 8 — Data layout (deferred).
Contiguous slices per cone and PSD size group; waits for a profile that justifies giving up bitwise comparability.

Phase 9 — Second-order cones (deferred until needed).
As in Section 8, with generator tests and MATLAB `K.q`/`K.r`.

Phase 10 — Documentation and release.
README rewrite, Doxygen, examples on the new options, migration notes for the old MEX signature and the API default changes, tagged release.

Milestones and effort. Milestone 1 is Pre-0 to 4b, including 2b; Phases 5 to 10 are re-planned once it lands. Milestone 2 is Phases 5–6b (cones, free variables, PSD), milestone 3 is Phase 7. The reorganization touches about 9,200 library and MEX lines, 2,900 submodule lines and 6,700 test lines [19]; the new readers and writers of Phase 2b add an estimated 3,000–4,000 lines including tests. The solver stays single-GPU. Neither repository has a license yet, and CHOLMOD includes GPL-licensed modules (O4).

Risks and mitigations

- *Silent behavior changes* — bitwise acceptance on each pinned machine for the structural phases; every intended change gets its own pull request, option value and golden files.
- *The default penalty policy loses problems* — Phase 4b gates against both the old rule and fixed σ; `legacy_adaptive` and `fixed` stay selectable.
- *Memory regressions* — the setup-time memory plan is tested against measured peaks.
- *cuDSS availability and version drift* — optional dependency with a pinned version; CHOLMOD remains the guaranteed backend.
- *Performance regressions from abstraction* — virtual calls happen once per component per iteration, not per element; the 2% per-iteration threshold runs in CI.
- *Reproducing published results* — `legacy_adaptive` and tagged, published commits.

18 Decisions

18.1 Adopted from the review

Q1. Penalty history. `legacy_adaptive` = `9bb4ba5`, kept through the migration as the bitwise baseline and Phase-4b comparator; the October 13 rule is `residual_balancing`; the default must have finite σ variation: `balance_freeze`, tuned and adopted in Phase 4b.

Q2. Low-rank start. Per block: from iteration 2 for blocks without a warm-up, as in today's CLI; from the end of the warm-up for blocks with one, as in today's C++ API.

Q3. Warm-up. FP16 only: `composite_fp16` by default for large blocks with $n \geq 5000$, none below; switched on in Phase 6a. The FP32 and FP32-emulated filters are not ported. This replaces the review's recommendation (`none` until Phase 6 shows a gain); the Phase-6a benchmark confirms or moves the threshold.

Q4. τ**.** Keep 1.618. Sweep $\{1.618, 1.9, 1.95\}$ at fixed σ, then re-screen the penalty at the chosen τ; avoid $\tau \geq 1.99$ from cold starts. 1.918 stays available.

Q5. Hybrid mode. Decide after plain ADMM with $\tau = 1.95$; probably retired.

Q6. MATLAB. `cuadmm.solve(At, b, C, K, opts)` with a positional shim for one release.

Q7. cuDSS. Optional, pinned minimum version, runtime check, deterministic mode under `strict`; admitted to `auto` after the benchmark.

Q8. Python. After Phases 2 and 4; public types without raw pointers.

Q9. `psd_projection`**.** Tagged and frozen; the submodule is removed after Phase 6a's parity tests.

18.2 Still open

O1. Default policy before Phase 4b: `residual_balancing` (the behavior of `main` since `2d8a755`, proposed) or `legacy_adaptive` (the goldens and published runs).

O2. Parameters of `balance_freeze`: decided by the Phase-4b screen (Table 2), including the factor (2 or 1.2) and the step schedule.

O3. Citation for $\tau \in (0, 2)$**.** [3] cites Gabay–Mercier for two-block ADMM "when one objective block is linear"; the review reads Gabay–Mercier as assuming strong convexity of one block. This plan relies only on [3] and [7]; whether the citation in [3] needs adjusting is for its authors to decide.

O4. License for cuADMM, compatible with CHOLMOD's modules.

O5. PSD blocks above the EVD limit ($n > 32{,}768$ on CUDA 12.9): reject (Phase 1), or later a CPU LAPACK fallback for the periodic exact projections. Multi-GPU eigensolvers are out of scope.

O6. Exact free equalities in the y-step (Section 9): evaluate on SOS problems after Phase 7.

O7. Preconditioned iterative solves (Section 11.6): priority and first preconditioner.

O8. Gap-aware vote: comparing weighted η_p and η_d so that the duality gap is balanced too [18]; untested.

O9. SPOT hand-over (Section 12): SPOT saves `sense` next to `prob`, and writes the cuADMM HDF5 file directly for large instances; for the SPOT maintainers.

A Response to the review of October 20

Review item	Revision 2	Where
Restore `legacy_adaptive`; name the edited rule; record the policy	Pre-0; policy table; summary contents	§7.3, §17
Default penalty must have finite σ variation; Phase 4b with CPU screen and GPU gates	balance-then-freeze rule; Phase 4b gates and budget	§3.5, §17
Publish the generator, push and tag commits, fix dangling references	Pre-0 housekeeping, owner named	B26, §17

Review item	Revision 2	Where
Retitle §3.5 and state its limits	rewritten as "The penalty parameter"	§3.5
`ResidualBalancingPenalty` parameters; σ_0 clamp; checkpointed policy state; warm-start reset	interface and options	§7.3, Table 5
σ log and summary contents; 17 significant digits	reporting	§7.3, B25
Phase 0: fixed-σ baselines first; one baseline per machine	Phase 0	§17
Phase 4 sGS criterion	defined precisely	§17
§10.4 dense-solve rule omitted conditions	all conditions restated	§11.5
Presolve off and CHOLMOD during bitwise phases	stated	§11.5
Loop sketch passed pre-step residuals	post-step residuals	§7
Per-iteration time threshold; single-stream mode	CI and core	§16, §14
First milestone; defer Phases 8–9; effort, single GPU, license	milestones paragraph; O4	§17
Run the benchmark early; cuDSS out of `auto` until then	selection rule; Q7	§11.5
No σ rule reads sGS residuals; hybrid confounded by τ	design box; Q5	§3.2
Keep the external validator independent	own component, host reference measures	§7.4
Return best and final iterate; ranking by DIMACS error	`BestIterateTracker`, `SolveResult`	§7.4, B23
Unflagged API default changes; option names; missing options	listed; `win_ratio`, `residual_weight`; options added	§15
Gabay–Mercier citation	removed from the claim; open point O3	§3.2
Lanczos value is an estimate; norm test to Phase 1	reworded; B22	§10.8
"Follows the convention of SDPNAL+"	reworded to the counter idiom	§3.5
Stale statements (Appendix A of the review)	corrected: commit status, phase count, 23 parameters, B4 and B5 scope, B6 step, KKT evaluator layer, CHOLMOD availability, EIG_FP64 scope, σ_0	throughout

Current	New	Note

B Mapping from current files to the new layout

Current	New	Note
`SDPSolver` (`solver.h`, `solver.cu`)	`Solver` facade + `admm/`, `cone/`, `linsys/`, `termination/`, `validation/`, `report/`	split
`SDPSolver::init`	`Solver::setup`: `problem/`, `scaling/`, `linsys/` factory, `ConeProduct` planning, memory plan	
`SDPSolver::solve` (loop)	`admm/AdmmSolver::run`	
y-step code, `solve_AAt_perm`	`AdmmState::y_step`, `linsys/direct/cholmod/`	
`large_full_eig_project`	`large/projectors/FullEvd` + `RankAnalyzer`	SYRK in 6b
`large_lobpcg_project`	`large/projectors/Lobpcg`	arena in 6b
`large_composite_project`	`large/projectors/CompositeFilter`	
medium and small tiers (inline)	`batched/`: `BatchedEvd`, `StreamedEvd`, `JacobiBatchedEvd`	
`cone_measures`,	`Cone::measure`,	
`certified_lower_bound`	`termination/Certificate`	
`validate_current_iterate`,	`validation/ExternalValidator`,	independent
`external_validation.*`	`ValidatedRule`	
σ update in `solve`, `monitors.h`	`admm/PenaltyPolicy`	`Monitor1` only in `LegacyPenalty`
best-iterate fields	`BestIterateTracker`	
`info_*` arrays, `sigma_log`, phase timers	`report/` observers	
`memory.h`	`core/device_buffer.h`, `core/handles.h`, `linalg/sparse_matrix.h`	
`check.h` (both)	`core/check.h`	one set of macros
`cublas.h`, `cusparse.h`, `cusolver.h`	`linalg/` wrappers, projector files	
`kernels/*.cu`	`linalg/vector_ops.cu`, `cone/psd/svec.cu`, `linsys/permutation.cu`	
`kernels/type_conversion.cu`	`matlab/`	MEX index conversion
`block_structure.*`, `matrix_sizes.*`	`problem/BlockLayout`, `cone/psd/batched/` layouts	
`utils/analyze_blk.cu`, `utils/get_maps.cu`, `MatrixSizes::init`, `utils/get_eig_rank_mask.cu`, `utils/check_gpus.cu`, `max_dense_vector_zero_mask`	—	deleted (unused)
`rank.cu`	`cone/psd/large/RankAnalyzer`	
`cholesky_cpu.h`	`linsys/direct/cholmod/CholmodSolver`	
`eig_cpu.h`	`test/` helper	test-only

Current	New	Note
`io.*`, `problem.*`	`io/`, `problem/`	
`main.cu`	`apps/cli/`	
`MATLAB/cuadmm_MATLAB.cu`	`matlab/`	
`psd_projection/src/composite_FP16.cu`	`cone/psd/kernels/composite_fp16.cu`	
`psd_projection/src/composite_FP32.cu`, `composite_FP32_emulated.cu`	—	not ported (Q3)
`psd_projection/src/lanczos.cu`	`cone/psd/kernels/lanczos_estimate.cu`	only the norm estimate
`psd_projection/src/lobpcg.cu`	`cone/psd/kernels/lobpcg.cu`	
`psd_projection/src/utils.cu`	`cone/psd/kernels/precision.cu`, `symmetrize.cu`	
`psd_projection/src/eig_FP64_psd.cu`, `eig_FP32_psd.cu`	`FullEvdFp32` (optional)	
`psd_projection/test/*`	`test/unit/psd/`	

C Notation and code names

Symbol	Meaning	Code name
X, y, S	primal variable, dual multiplier, dual slack	`X`, `y`, `S`
X_u, S_u	free block and its (zero) dual slack	`FreeCone`
$\mathcal{A}$, $\mathcal{A}^*$	constraint map and adjoint	`A`, `At`
$\mathcal{A}\mathcal{A}^*$	normal matrix	`NormalSystem`, `NormalOperator`
$\mathcal{K}$, $\mathcal{K}^*$	cone and dual cone	`Cone`, `project_dual`
σ, τ	penalty, step length	`sigma`, `tau`
(2), (3), (4)	the three steps	`y_step`, `cone_step`, `multiplier_step`
$\eta_p, \eta_d, \eta_g, \eta$	residuals	`eta_p`, `eta_d`, `eta_g`, `eta`
w_p, w_d, K_f	win counters, freeze iteration	`ResidualBalancingPenalty`
ν, b_s, C_s	scaling factors	`Scaling::row_norms`, `b_scale`, `c_scale`
M, W	large-block input buffer and shared workspace	`LargeBlockJob`
r_+, r_-, tol_r	ranks and rank tolerance	`RankAnalyzer`
ρ, R	low-rank ratio, re-evaluation period	`psd.lowrank.ratio`, `.reevaluate`

References

[1] A. Groudiev, S. Kang, and H. Yang. cuADMM: GPU-accelerated first-order optimization for large-scale multi-block semidefinite programs. In *RSS Workshop on Fast Motion Planning and Control in the Era of Parallelism*, 2025. https://agroudiev.github.io/assets/pdf/8_cuADMM_GPU_Accelerated_First.pdf.

[2] S. Kang, H. Han, A. Groudiev, and H. Yang. Factorization-free orthogonal projection onto the positive semidefinite cone with composite polynomial filtering. arXiv:2507.09165, 2025 (v2, 2026).

[3] S. Kang and H. Yang. Afterimage slow regions in first-order methods for linear conic programming. arXiv:2609.09449, 2026.

[4] S. Kang, X. Xu, J. Sarva, L. Liang, and H. Yang. Fast and certifiable trajectory optimization. In *International Workshop on the Algorithmic Foundations of Robotics (WAFR)*, 2024. arXiv:2406.05846.

[5] S. Kang, X. Jiang, and H. Yang. Local linear convergence of the alternating direction method of multipliers for semidefinite programming under strict complementarity. arXiv:2503.20142, 2025.

[6] L. Chen, D. Sun, and K.-C. Toh. An efficient inexact symmetric Gauss–Seidel based majorized ADMM for high-dimensional convex composite conic programming. *Mathematical Programming*, 161(1–2):237–270, 2017.

[7] L. Chen, X. Li, D. Sun, and K.-C. Toh. On the equivalence of inexact proximal ALM and ADMM for a class of convex composite programming. *Mathematical Programming*, 185(1):111–161, 2021.

[8] Z. Wen, D. Goldfarb, and W. Yin. Alternating direction augmented Lagrangian methods for semidefinite programming. *Mathematical Programming Computation*, 2(3):203–230, 2010.

[9] S. Boyd, N. Parikh, E. Chu, B. Peleato, and J. Eckstein. Distributed optimization and statistical learning via the alternating direction method of multipliers. *Foundations and Trends in Machine Learning*, 3(1):1–122, 2011.

[10] D. A. Lorenz and Q. Tran-Dinh. Non-stationary Douglas–Rachford and alternating direction method of multipliers: adaptive step-sizes and convergence. *Computational Optimization and Applications*, 74(1):67–92, 2019.

[11] B. Wohlberg. ADMM penalty parameter selection by residual balancing. arXiv:1704.06209, 2017.

[12] D. Sun, K.-C. Toh, Y. Yuan, and X.-Y. Zhao. SDPNAL+: A Matlab software for semidefinite programming with bound constraints (version 1.0). *Optimization Methods and Software*, 35(1):87–115, 2020.

[13] Y. Chen, T. A. Davis, W. W. Hager, and S. Rajamanickam. Algorithm 887: CHOLMOD, supernodal sparse Cholesky factorization and update/downdate. *ACM Transactions on Mathematical Software*, 35(3), 2008.

[14] NVIDIA Corporation. cuDSS: NVIDIA CUDA direct sparse solver library, documentation. https://docs.nvidia.com/cuda/cudss/.

[15] R. J. Vanderbei. Symmetric quasidefinite matrices. *SIAM Journal on Optimization*, 5(1):100–113, 1995.

[16] N. Amsel, D. Persson, C. Musco, and R. Gower. The Polar Express: optimal matrix sign methods and their application to the Muon algorithm. arXiv:2505.16932, 2025.

[17] A. V. Knyazev. Toward the optimal preconditioned eigensolver: locally optimal block preconditioned conjugate gradient method. *SIAM Journal on Scientific Computing*, 23(2):517–541, 2001.

[18] Does the adaptive σ rule on `main` fit cuADMM's plain ADMM? Internal report on the penalty update at `2d8a755`, October 19, 2026.

[19] Is the reorganization on cuADMM `main` reasonable? Internal review of revision 1 of this plan, October 20, 2026.